

سجل المحادثات — zzz.txt

الجزء 7 من 11

الصفحات 499-584 من أصل 913 • 10 محادثة

gpt-5.5-instant — 92	499
gpt-5.5-instant — 93	512
gpt-5.5-instant — 94	514
gpt-5.5-instant — 95	518
gpt-5.5-instant — 96	533
gpt-5.5-instant — 97	544
gpt-5.5-instant — 98	561
gpt-5.5-instant — 99	568
gpt-5.5-instant — 100	570
gpt-5.5-instant — 101	571

strict causal stream order, symmetric vs ordered same-batch semantics, unordered tie-set with canonical serialization, witness-gated closed-domain absence, and OBSERVATION_ADJACENT $\neq$ GRID_CONTIGUOUS.

Therefore: ACCEPTED — meets contractual closure criteria without modification. The S1 artifact at commit bd2fe68 with hashes above is the audited, PENDING-AUDIT $\rightarrow$ ACCEPTED baseline for MUF S1.

AUDIT ARTIFACTS

This report:

MUF_S1_INDEPENDENT_AUDIT_READONLY.md

Prior guard re-audit:

FIELD_RUNNER_MUF_S0_BASELINE_GUARD_REAUDIT.md

(177 lines, guard c7e6cf8c..., MANIFEST 899febb4... 10/10 probes PASS)

Clone path: /tmp/trading_bot_latest (expired after audit; hashes above are the canonical pins)

Independent attack harness: /tmp/s1_audit_checks3.py (8 targeted probes, all FAIL-CLOSED)

Read-only: no file in project/trading_project was modified, no MANIFEST was re-hashed, no deliverables/ besides this report.

gpt-5.5-instant

الحالة الرسمية: S1 صار IMPLEMENTED — PENDING AUDIT. البيلدر أضاف 4 ملفات فقط، 68 اختبار S1، والمشروع صار 1207/1207، S0 بقي 67/67، Field Runner 36/36، والMANIFEST بقي 196/196 ببصمة 899febb4...90e84. ما رح نرجع للتصميم الآن؛ الخطوة الصحيحة Audit تنفيذي واحد شامل.

لكن من تقرير البيلدر نفسه في عدة مناطق لازم الأوديتور يضربها، لأنها قد تكون أخطاء حقيقية رغم ال1207 اختبار الأخضر. أهم واحدة: البيلدر يقول:

key_axis استنتاجي من وجود timestamp فقط

هذا يرفع عندي راية حمراء. S0 عنده axis semantics صريحة. ما بدى S1 يستنتج:

[text]

timestamp موجود $\rightarrow$ TIME_INDEXED

timestamp غائب $\rightarrow$ POSITIONAL

إذا الInformationKey/Timeline contract يحمل axis authority بطريقة أخرى. لازم Audit يثبت أنه يستخدم السلطة العامة الصحيحة، مش heuristic.

الثانية: RC1 طلب schema foundations الCausalEpisodeRecord و الEpisodeMembershipEvent و الTransition Explanation. التقرير من جهة يقول "4 schemas فقط: adjacency..." ومن جهة يقول "episode anchor $\neq$ membership (لا حلقة episode في S1 نهائياً)". لازم نتأكد هل schema-only foundations المطلوبة موجودة فعلاً أم تم إسقاطها بالخطأ لأنه فهم "لا population" = "لا schema". إذا غير موجودة، هذا Contract Scope Miss.

الثالثة: غياب grid. التقرير يقول إن ExpectedGridAbsenceWitness يتحقق من tokens مشهّدة من Stage 4C-1 لكنه "لا يعيد حساب المقارنة". ممتاز إذا كان يستهلك closed public authority فعلاً. سيئ جداً إذا كل ما يحتاجه هو string: token

[text]

"GRID_OBSERVATIONS_MISSING"

من caller. وقتها أي واحد يقدر يزور شهادة الغياب. لازم provenance binding حقيقي للمصدر/الداتا/الtimeline، مش token laundering.

الرابعة: same-batch. التقرير يقول signed displacement والاتجاه يصبحوا UNDEFINED داخل same batch، بينما absolute close-path step يبقى EXACT. هاي ممكن تكون صحيحة رياضياً إذا القيمتان co-visible لكن لا نعرف chronology: |C2-C1| نفسها لا تعتمد على أيهما أول، لكن هل تسميتها "path step between adjacent observations" ما زالت شرعية إذا ما في order؟ لازم نفرق بين symmetric distance وبين directed adjacency fact. الأوديتور لازم يحسم.

الخامسة: frozenset للpayload. S0 freeze_payload كان يرفض set/frozenset كpayload. إذا RunningExtremeState داخلي فقط ممكن تمام. إذا ينشر frozenset مباشرة داخل PublishedRecord، رح يضرب contract أو يعمل canonicalization موازي. لازم نهاجمه.

MUF V1 S1 — INDEPENDENT IMPLEMENTATION AUDIT
READ ONLY — SAME ARTIFACT — NO FIXES

Reference:

MUF S0:
CLOSED

Official baseline before S1:

MANIFEST SHA256:
899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84

MANIFEST:
196/196 OK

Historical project tests:
1139/1139

Field Runner:
36/36

S1 design authority:
RC1
→ H1
→ S1 FINAL IMPLEMENTATION DESIGN

Builder:
MUF S1 IMPLEMENTED — PENDING AUDIT

Builder reports:

S1 dedicated:
68/68

S0:
67/67

Full project:
1207/1207

Field runner:
36/36

S1 files intentionally unmanifested pending audit.

AUDIT RULE:
Do not trust builder tests/report.
Attack implementation.

One reproducible contract violation is enough for PATCH REQUIRED.

No modifications.
No MANIFEST.
No S2.

A — ARTIFACT / SCOPE GATE

Determine exact SHA256 of all four S1 files from the actual artifact and report them.

Verify:

- exactly four authorized S1 files added.
- zero pre-existing src/tests files modified.
- S0 byte-identical.
- docs unchanged.
- field_runner unchanged.
- MANIFEST remains exact baseline:

```
899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84
```

- existing 196 entries all OK.

Any mismatch:

REJECT ARTIFACT / OWNER REVIEW.

B — PUBLIC API / AXIS AUTHORITY

Inspect all S1 imports and axis logic.

Mandatory question:

Does S1 derive POSITIONAL vs TIME_INDEXED merely from presence/absence of event_time_utc?

Builder report says:

"key_axis inferential from timestamp presence."

Determine whether this is fully equivalent to the CLOSED public InformationKey axis contract.

Attack malformed/edge keys where:

- timestamp presence conflicts with actual timeline/axis semantics,
- positional timeline receives timestamp,
- time-indexed timeline lacks timestamp,
- different timeline adapters exist.

Expected:

S1 must use authoritative public axis semantics,
not invent a weaker heuristic.

If timestamp-presence inference can misclassify a legally constructible key or
bypass axis authority:

BLOCKER.

No private CLOSED APIs allowed.

C — PRICE SOURCE SEMANTICS

Verify canonical price fields are typed only as PUBLISHED_OHLC_FACT.

Search production S1 for ACTUAL/PROXY.

Allowed:

optional flow-context semantics only if actually implemented by contract.

Forbidden:

price field labelled ACTUAL/PROXY.

Attack forged provenance:

- source_identity mismatch

- dataset_identity mismatch
- published_bar_ref wrong schema/type
- cross-timeline provenance

Expected fail closed.

Verify S1 is not Binance-hardcoded except through currently integrated public source contract.

D — OHLC VALIDITY / INTRABAR AMBIGUITY

Attack:

- 1) $H < L$.
- 2) $H < 0$.
- 3) $H < C$.
- 4) $L > 0$.
- 5) $L > C$.

6) NaN/inf.

7) source-illegal nonpositive price where source contract forbids it.

No repair/clipping.

Then attack chronology:

- ask H-before-L.
- ask exact intrabar path.
- inspect any formula/API that implicitly assumes traversal.

Expected:

exact intrabar path UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY).

Recompute lower-bound proof independently on:

- discrete jump sequences

- $O==H$
- $O==L$
- $C==H$
- $C==L$
- $H==L$

- random admissible sequences.

Verify $TV \geq LB$.

No continuity claim anywhere.

E — STREAM ORDER / ADJACENCY

Independently attack:

- 1) increasing legal keys.
- 2) duplicate key.
- 3) out-of-order comparable key.
- 4) incomparable key.
- 5) cross-timeline.
- 6) cross-axis.
- 7) shuffled rows.
- 8) same-batch keys with differing deterministic_sequence.

No silent sort.

State before failed accept must remain unchanged.

OBSERVATION_ADJACENT must mean only consecutive accepted observations under the lawful stream contract.

It must not automatically imply GRID_CONTIGUOUS.

F — SAME-BATCH SEMANTICS

This is mandatory.

Builder reports:

- signed displacement/direction = UNDEFINED when chronology unproven.
- absolute close path step remains EXACT.

Test whether implementation distinguishes:

A)

symmetric metric:

$|C_a - C_b|$

from

B)

ordered fact:

"from a to b" / signed displacement / direction / observation adjacency.

If same-batch chronology is unknown:

the symmetric distance may be exact as a set-distance,
but must NOT be mislabeled as an ordered market path traversal.

Audit record names/metric_semantics/provenance.

If absolute metric is published as an ordered CLOSE_PATH_STEP requiring

A-before-B without proven chronology:

BLOCKER or require neutral symmetric naming.

Do not reject symmetric arithmetic merely because chronology is unknown.

G — RUNNING EXTREMES / TIES

Attack:

- new high.
- equal high.
- new low.
- equal low.
- all highs equal.
- all lows equal.
- input representation permutations for same legal facts.

Verify:

all tied origins retained.

No:

first

winner

true extreme

row-order tie-break.

deterministic_sequence cannot select semantic winner.

Mandatory S0 compatibility:

Builder reports frozenset internally.

Check whether frozenset ever enters:

PublishedRecord/EventRecord payload
or canonical S0 freeze_payload path.

S0 payload domain rejects set/frozenset.

If S1 publishes a frozenset directly in S0 persisted payload:
BLOCKER.

Internal transient frozenset is acceptable only if converted to an accepted
canonical immutable payload representation without semantic ordering claim.

H — GRID / ABSENCE AUTHORITY

Attack ExpectedGridAbsenceWitness aggressively.

Builder says it validates Stage 4C-1 certified tokens without recomputing.

Prove this cannot be forged by caller-controlled strings alone.

Mandatory:

1) no declared grid / no sealed authority.

Attempt absence.

→ no factual absence.

2) caller supplies literal string/token:

GRID_OBSERVATIONS_MISSING

without valid CLOSED provenance.

→ must fail.

3) valid Stage 4C-1 public coverage artifact for sealed D1 says key absent.

→ factual domain-scoped absence allowed.

4) source identity mismatch.

→ reject.

5) dataset identity mismatch.

→ reject.

6) timeline mismatch.

→ reject.

7) cadence/grid mismatch.

→ reject.

8) coverage domain does not include expected key.

→ reject.

9) D1 absence then corrected D2 contains key.

→ D1 record unchanged; D2 distinct identity.

10) coverage UNKNOWN/UNAVAILABLE.

→ no absence fact.

Critical:

If Stage 4C-1 authority is consumed through private symbols:

BLOCKER.

If public contract lacks enough provenance to authenticate the absence witness:

zzz.txt — سجل المحادثات

BLOCKER.

Do not recreate private semantics locally.

I — GRID CONTIGUITY / GAP SEMANTICS

Verify:

OBSERVATION_ADJACENT

≠

GRID_CONTIGUOUS.

For observed positions/times:

10:00,10:01,10:03

without authority:

no factual absence record.

With lawful closed-domain authority:

expected key 10:02 not observed in D1 may be recorded domain-scoped.

Grid-contiguous accumulator must not bridge a missing expected observation as if continuous.

No forward fill.

No synthetic bar.

No "market bar absent" wording/field.

J — SCHEMA SCOPE COMPLETENESS

This is mandatory.

Accepted S1 design requires schema-only foundations for:

CausalEpisodeRecord

EpisodeMembershipEvent

MarketStateTransitionRecord

ExplanationRecord

Builder report is ambiguous:

it says "4 schemas only: adjacency / grid-contiguity / pair-metric-bundle / displacement-with-direction"

and also says episode anchor ≠ membership.

Inspect actual path_schemas.py.

For each required accepted schema answer:

IMPLEMENTED

or

MISSING.

Requirements:

CausalEpisodeRecord:

anchor/creation facts only;

NO membership_refs.

EpisodeMembershipEvent:

separate append-only schema.

MarketStateTransitionRecord:

literal factual state/descriptor transition only;
no regime thresholds.

ExplanationRecord:

allowed states exactly:

MONITORING

PATTERN_REQUIREMENTS_SATISFIED

CONTRADICTED

SUPERSEDED.

No:

probability

weight

score

support.

S1 need not populate these.

But accepted design requires the SCHEMAS.

If required schemas are absent:

BLOCKER — scope implementation miss.

Do not accept "population is NOT_CONFIGURED" as excuse for missing schema foundation.

K — DESCRIPTOR SPECS

Inspect all seven.

Verify:

- RUNNING_ONLY where designed.
- required inputs legal.
- policy_dependencies empty.
- no hidden numeric window.
- no final-wave/future input.
- zero denominator typed UNDEFINED.
- no epsilon.
- no inf.
- boundary contract explicit.

Attack registration with:

- policy dependency.
- undeclared boundary.
- fixed hidden window.
- final future input.
- unsupported descriptor name/schema.

Fail closed.

L — FUTURE APPEND / REPAINT

For a legal prefix T:

capture canonical representations of every published S1 output $\leq T$.

Append legal future observations.

Recompute/query old prefix.

Expected old outputs identical for:

- pair facts
- derivatives
- running extreme snapshots
- descriptor snapshots
- lawful absence records scoped to sealed D1
- schema-only records if instantiated in fixtures.

Test future:

- new high
- new low
- equal tie
- gap
- corrected dataset D2.

No mutation/repaint.

M — PERFORMANCE

Do independent source inspection and deterministic operation-count attacks.

Builder claims:

$O(1)$ amortized/bar

$O(n)$ total.

Run scaling:

N , $2N$, $4N$.

Attack:

- primitive processing
- running extremes
- descriptors
- grid handling

No hidden history rescan per bar.

Tie-origin memory $O(k)$ is allowed.

No arbitrary truncation.

If any normal legal all-bar processing path is $O(n^2)$:

BLOCKER.

N — FORBIDDEN ONTOLOGY / SCOPE

AST/search production code independently for executable logic implementing:

turning point

swing

wave

segment/run ontology

hierarchy

α/β/γ/δ

policy fit
calibration
model
strategy
signal
PnL
OOS
narrative population
statistical independence.

Do not flag comments/test guards merely naming forbidden concepts.

DisplacementRun/segment implementation:
must be absent.

O — MUTABILITY / S0 RECORD COMPATIBILITY

Instantiate all S1 persisted/public record forms.

Verify they use S0 immutable payload semantics.

Attack:

- top-level mutation
- nested mutation
- caller alias mutation
- unsupported custom object
- set/frozenset persistence
- future append.

No parallel mutable record implementation.

P — ERROR CONTRACT

Verify deterministic distinctions at least for:

S1_INVALID_OHLC
S1_OUT_OF_ORDER_OBSERVATION
S1_DUPLICATE_OBSERVATION_KEY
S1_ABSENCE_REQUIRES_CLOSED_DOMAIN
S1_GRID_CONTIGUITY_UNPROVEN
UNSUPPORTED_SOURCE_SEMANTICS
AMBIGUOUS_INTRABAR_CHRONOLOGY
SAME_BATCH_ORDER_UNPROVEN
ZERO_DENOMINATOR

No broad catch converting programmer bugs into valid typed missingness.

Q — INDEPENDENT MUTATION PROOFS

Do not trust builder's A-O.

At least independently mutate isolated copies for:

- 1) infer H-before-L.

- 2) silently sort out-of-order keys.
- 3) choose first tie origin.
- 4) accept absence from literal token without provenance.
- 5) add membership_refs to episode anchor OR remove required schema.
- 6) deterministic_sequence creates direction in same batch.
- 7) restore $O(n^2)$ history rescan.
- 8) allow frozenset persisted through S0 payload.

Relevant gates must fail.

No residue.

R — TEST / NON-TOUCH

Run:

S1 dedicated:
expected 68.

S0:
67.

Full:
1207.

Field runner:
36.

MANIFEST:
196/196 existing entries OK
sha unchanged.

Audit modifies nothing.

S — INDEPENDENT ATTACKS

Add at least 10 attacks not copied verbatim from builder.

Prioritize:
axis inference,
absence witness forgery,
schema omission,
same-batch symmetric-vs-directed semantics,
tie persistence,
future append,
source/dataset provenance,
complexity.

Do not invent blockers.

T — DECISION

Return exactly one:

- 1) ACCEPTED FOR CLOSURE
- 2) PATCH REQUIRED
- 3) REJECTED — DESIGN REQUIRED

For every BLOCKER:

- contract violated
- exact file/function/schema
- minimal fixture
- reproduction
- expected
- actual
- smallest local correction
- whether implementation patch or design issue

NON-BLOCKING:

separate.

If ACCEPTED:

report exact four S1 hashes,
S1/S0/full/field_runner counts,
MANIFEST identity,
complexity result,
absence-authority result,
schema-completeness result,
same-batch result.

READ ONLY.

.STOP

هلق ما منعمل أي تعديل إضافي من عندنا. نستنى نتيجة الأوديتور.

إذا طلع مثلاً أن schema foundations الأربعة ناقصة، هذا Patch واحد واضح. إذا طلع absence witness قابل للتزوير، هذا Blocker حقيقي. إذا كل هالهجمات مرت، منغلق S1 وننتقل.

بهاطريقة حافظنا على الدقة بدون ما نرجع نعمل خمس جولات تصميم جديدة.

لا ماعجبتي خلص بيلدر واحد يلي عم نشغل عليه

هيك جاويني البيلدر :

MUF V1 S1 BUILD REPORT (P1 → S0 → S1) — 56

الحالة: IMPLEMENTED — PENDING AUDIT

(إحدى الحاليتين المسموحتين فقط — لا إغلاق بلا موافقة المالك؛ ACCEPTED ≠ BUILD AUTHORIZED)

التاريخ: 01-10-2026

المُرسل: P1 Builder (حزمة البناء فقط)

يلغيان نطاقهما الصريح RC1/H1 → 53 → 54 → 55: RC1 → H1 → S1 FINAL IMPLEMENTATION DESIGN (المصدر المُلتزم فقط)

نطاق المهمة (23): 4 — BUILD ONLY ملفات جديدة فقط · صفر تعديل على موجود · بلا MANIFEST/docs/S2

المخرجات (الملفات الأربعة فقط — unmanifested)

الملف الدور الأسطر (تقريبي)

PUBLISHED_OHLC_FACT · src/trading_system/market_understanding/price_path.py

LB · P1-P7 المتقطع · stream · tie-sets · RUNNING_EXTREME صارم · ~800 MetricResult · DescriptorSpec(7)

src/trading_system/market_understanding/path_schemas.py طبقة السكيمات: حدث canonical · غياب الشبكة

بشهادة cadence partition · RC1 · تطبيع inputs · سجلات الأزواج · ~640 checkpoints

tests/test_muf_s1_price_path.py بوابات الوحدة 1-20 اختباراً

tests/test_muf_s1_path_schemas.py بوابات 1-20 + 48 A-O MUT + A-G H1 اختباراً

MANIFEST: لم يُمس — 196 سطرًا ·

84e8409162185d4d546bf6fe972405a97176b48198433a4bfeb899 · كل المدخلات OK. ملفات

S1 الأربعة unmanifested عمداً (S0 مجمّد).

عدّادات البوابات الفعلية (gate 17)

البوابة النتيجة

اختبارات S1 الجديدة (الملفان) 68/68 (20 + 48)

بوابة S0 (test_muf_s0 * الأربعة) 67/67

المجموعة الكاملة S1 1139 baseline + tests/ 1207/1207 =

field_runner/runner_tests 36/36 (home/user/ من)

ماسحات S1 source (private / prohibited / market_shape) 0 / 0 / 0

أرقام إنتاجية داخل defs بملفا {0,1} S1 فقط (AST sweep = صفر خروج)

بوابات الطفرات A-O على نسخ معزولة 15/15 تستهدف ضعفها المحدد + الضابط ينجح

NON-TOUCH (قبل/بعد بكل SHA) 0 تعديل 0 حذف 4+ الملفات المسموحة فقط

ملاحظة قياسية: scan_market_shape على ملفات الاختبارات يُرجع أرقام fixtures — وهذا هو السَّبق المُثبت (ملفات S0 القائمة كذلك، مثلاً test_muf_s0_records = 81 finding) القاعدة الرقمية على كود الإنتاج وملفا S1 source نظيفان تماماً.

حصر public API المكتوب (gate 15)

price_path.py (اسم → دور):

ثوابت مفردات: EXACT/BOUND/PROXY/UNAVAILABLE · AMBIGUOUS_INTRABAR_CHRONOLOGY ·

UNBOUNDED_REFINEMENT · NOT_TIME_INDEXED · ZERO_DENOMINATOR ·

SAME_BATCH_ORDER_UNPROVEN · GRID_OBSERVATION_NOT_PROVEN_CONTIGUOUS ·

DISCRETE_TV_TRIANGLE_INEQUALITY · DIRECTION_UP/DOWN/FLAT ·

S1_* · PUBLISHED_OHLC_FACT · أكواد الرفض · ADJACENCY_OBSERVATION_ADJACENT/GRID_CONTIGUOUS

PRICE_PATH_SCHEMA · STAGE_* · BOUNDARY_SINCE_GENESIS/ORIGIN ·

AVAILABILITY_AT_ACCEPTED_KEY · S1_DESCRIPTOR_INPUT_KINDS

دوال: MetricResult · exact_metric · bound_metric · unavailable_metric · metric_payload · safe_ratio ·

key_serialization · key_axis · pair_bar_count_duration · pair_wall_clock_duration · PublishedOhlcBarFact

(+ .provenance) · canonical_origin_order · RunningExtremeState (+ .origins_unordered · .seed ·

.observe) · intrabar_path_metrics · bar_derivative_metrics · same_batch_chronology_unproven ·

pair_metrics · pair_direction · pair_adjacency_kind · DeclaredGrid · DescriptorSpec ·

S1_DESCRIPTOR_SPECS · DESCRIPTOR_SPECS_BY_NAME · AcceptedBarFacts · CausalObservationStream

(+ accept · accepted_count · accepted_keys · last_accepted_key · observed_close_path_length ·

grid_contiguous_close_path_length · running_high_state · running_low_state · dataset_identity)

:path_schemas.py

ثوابت: S1_SCHEMA_IDENTITY · S1_OBSERVATION_GRID_KEYS ·

S1_EXPECTED_GRID_KEY_ABSENCE_RECORD · S1_OBSERVED_ADJACENCY_DESCRIPTOR ·

S1_GRID_CONTIGUITY_DESCRIPTOR · S1_PAIR_METRIC_BUNDLE ·

S1_PAIR_DISPLACEMENT_WITH_DIRECTION · EXPECTED_GRID_KEY_NOT_OBSERVED ·

S1_TYPED_STATE_UNAVAILABLE · S1_NOT_YET_OBSERVED · S1_MISSING_STATES ·

STAGE_4C1_AUTHORITY_MODULE · إعادة تصدير tokens المشرح (6)

دوال: require_s1_schema_version · bar_provenance · observe_complete_bar_event ·

ExpectedGridAbsenceWitness · absence_pair_tuple · StreamCheckpoint · capture_checkpoint ·

verify_checkpoint_continuity · ExactSlotCadence · s1_scheduled_slot_coverage · cadence_partition ·

DescriptorInputState · normalize_descriptor_inputs · PairSchemaRecords · pair_path_schemas ·

expected_grid_absence_record · observed_grid_keys_record

كل استيرادات S1 من public S0 modules فقط (فاحص الاستيرادات الخاصة = صفر)؛ لا تعريف متوازٍ لأي عقد S0.

→ التنفيذ 18-1 بنود التصميم

غير عبر PUBLISHED_OHLC_FACT + provenance: PublishedOhlcBarFact (source_identity + dataset_identity + axis

مخترع · لا timestamp · لا S1_INVALID_OHLC) clipping صارم بلا OHLC تحقق · (published_bar_ref + المفتاح

ترميز 24×7.

public semantics المقارنة عبر S1_OUT_OF_ORDER_OBSERVATION · S1_DUPLICATE_OBSERVATION_KEY / صارم Stream

S0 (InformationKeyError/IncomparableInformationKeys) يفشل مغلقاً بعقد incomparable · (key > previous) فقط

· يثبت عدم تغيّر الحالة 12 بلا فرز خفي (الاختبار ·

P1-P7 + UP/DOWN/FLAT : displacement (signed) · close_path_step · bar_range · open_close_displacement

· wicks · bar_count · bar_count_on_grid – والاتجاه مفردات مغلقة

LB المتقطع TV ≥ (H-L) + min(|0-H|+|L-C|, |0-L|+|H-C|) بمرجع DISCRETE_TV_TRIANGLE_INEQUALITY · EXACT =

UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY) · upper = UNAVAILABLE(UNBOUNDED_REFINEMENT) · fixture

(10,15,5,12→(18) و 10,12,8,11→(7) الهينات:

tie-sets التخزين canonical winner: RunningExtremeState.origins_unordered (frozenset) غير مرتبة بلا

serialization (موتّق non-semantic) فقط MUT-E/J) في 50/50) تحتفظ بكل الأصول ties ·

الاسم الوحيد) مُشّهدة tokens يتحقق من ExpectedGridAbsenceWitness فقط Stage 4C-1 بسلطة grid غياب

trajectory_stage4c) ولا يُعيد حساب المقارنة إطلاقاً

POSITIONAL timestamp استنتاجي من وجود key_axis مخترع timestamp بلا POSITIONAL

UNAVAILABLE(NOT_TIME_INDEXED) على

zzz.txt — سجل المحادثات

EXACT • للإزاحة المُوقَّعة والاتجاه • القيمة المطلقة تبقى batch: UNDEFINED(SAME_BATCH_ORDER_UNPROVEN) بنفس الـ لا يُرقى أبداً UNKNOWN.

DescriptorSpec + 7 descriptors: running-only • policy_dependencies=() (مخالفة = S1_POLICY_DEPENDENCY_FORBIDDEN) • denominator مُطَبَّع مُغلف (UNDEFINED(ZERO_DENOMINATOR)) بلا epsilon/infinity.

4 schemas: adjacency • grid-contiguity • pair-metric-bundle • displacement-with-direction – recorded_kind/recorded_property بلا real/fake • episode anchor ≠ membership (لا حلقة) episode في S1 (نهائياً).

future-append invariance: البادئ prefix يثبتان تطابق سجلات H1-F + 20 الاختباران.

0(1)/bar • 0(n) • tie 0(k) (حساب بدائي واحد لكل زوج: monkeypatch = n-1) • rescan (MUT-J) • بلا truncation. لعينات التعادل.

بوابة طفرات مُثبتة أدناه. A=0: 15 المصفوفة الكاملة + بوابات طفرة (تتجاوز المطلوب الست) – كل واحدة تُسقط بوابتها المحددة 15: 6+ mutation proofs.

RESEARCH-DEBT-024 OPEN (لا ادعاء استقلال • private لا استيراد • gate 15) فقط: الحصر أعلاه public API (كما).

مجّمد. S0 • S1 files unmanifested • الكامل أعلاه SHA بال NON-TOUCH صفر تعديل: إثبات عدّادات فعلية: الجدول أعلاه أرقام فعلية لا مُتوقَّعة. (أعلاه) IMPLEMENTED – PENDING AUDIT. التقرير بحالة واحدة من المسموح /tmp/s1_mut_proofs/ – على نسخ معزولة) A=0 مصفوفة الطفرات الطفرة المستهدفة البوابة المُسقطَة (أمثلة) النتيجة

✓ مُتوقَّع FAIL 03 اختبار A min-max في LB mut_a + h1b +

✓ FAIL 08 + h1c اختبار B same-batch mut_b + إلغاء قاعدة

✓ FAIL 10/12/14 اختبار C mut_c + إلغاء فحص الترتيب

✓ FAIL 11 اختبار D mut_d + إلغاء فحص التكرار

✓ FAIL 15 اختبار E tie winner (استبدال عند التعادل) mut_e +

✓ FAIL 01 اختبار F OHLC mut_f + إلغاء قانونية

✓ FAIL 04 اختبار G mut_g + صفر مقام → رقم

✓ FAIL 16 اختبار H timestamps (TIME_INDEXED دائماً) mut_h + اختراع

✓ FAIL mut_i + rc1b1 إلغاء بوابة الشهادة I

✓ FAIL (49عدّاد) mut_j + h1e (rescan) إعادة مسح التاريخ J

✓ FAIL 17 اختبار K grid-contiguity زائفة mut_k +

✓ FAIL POSITIONAL mut_l على L wall-clock

✓ FAIL (حارس المفردات) mut_m (detect_swing_points) ممنوع S2 تعريف M

✓ FAIL off-grid mut_n يحتوي N grid path

✓ FAIL 09 اختبار O FLAT→UP mut_o +

✓ PASS الضابط (بلا طفرة) الملفان كاملان –

قرارات تنفيذية موثقة (لا تتعدى نطاقها) 6)

لا صنف أخطاء موازي – لا تعديل على) الحالي S0 وفق نظام SchemaViolation كـ S1_* يُصدر أكواد رفض S1: عدّاد الأخطاء S0 (مسموح S0).

يحدث عبر DIFFERENT_BATCH غير مثبتة؛ chronology: KNOWN_SAME_BATCH/UNKNOWN_IF_SAME_BATCH: الدفعة الواحدة الخاص. _comparison_tuple بلا استدعاء – (مثلاً causal_position) العامة S0 اختلاق إحدائيات – (EXPECTED_GRID_KEY_NOT_OBSERVED + typed UNAVAILABLE) متوقع غير مُشاهد grid الغياب: سجل واحد لكل مفتاح لا مطالبة «اكتمال بيانات» إطلاقاً.

(لا امتداد تنفيذي) multi-timeline المُدخل؛ لا معالجة timeline على S1 الخط الزمني الواحد: كل سجلات prefix. من استمرارية 0(1) فقط + تحقق increment يخزّن حالة StreamCheckpoint: bounded-memory checkpoints.

(ممنوع – لم يُنَ) S1 ما هو خارج 7)

• نافذة مخفية • calibration • policy • αβγδ • swing/turning-point/wave/segment/hierarchy • إنتاج • S2

(raw) « • بيانات المالك 24×7 ادعاء استقلال • «كريبتو • narrative • OOS • Model-Strategy-Signal-PnL • aggTrades/minuteFacts/sidebar) – كل الاختبارات SYNTHETIC SMALL FIXTURES ONLY.

إغلاق المهمة

حالة التسليم: IMPLEMENTED — PENDING AUDIT.

التدقيق READ ONLY في شات منفصل؛ أي هجوم واحد يمنع القبول؛ لا تختبر blocker.

لا إغلاق (S1 CLOSED) ولا التحقق التالي إلا بأمر المالك.

قناة GitHub (أمر دائم): commit + push على TRADING-BOT (خاص) — تم تنفيذه مع هذا التقرير.

STOP — BUILD COMPLETE (S1) — PENDING AUDIT.

gpt-5.5-instant

الحالة الرسمية: S1 جاهز BUILD، وS0 CLOSED. فكرة Builder ثاني ممتازة إذا استخدمناه للتوازي الحقيقي، وسيئة جداً إذا خيلنا A Builder وB يعدّلوا نفس الشجرة أو بينوا مرحلتين تعتمد الثانية على الأولى بنفس الوقت.

يعني نعم: اعمل Builder ثاني. لكن ما منقول «كل واحد يبني مرحلة وتركبهم». لأن مثلاً:

[text]

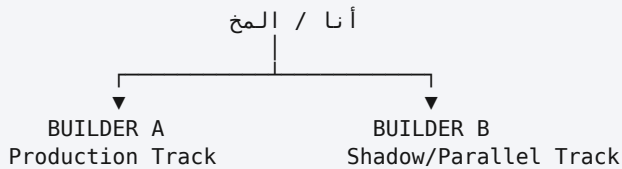
Builder A يبني S1

Builder B يبني S2 فوق تصميم S1 المتوقع

وبعدئ S1 يتغير بالAudit؛ صار S2 كله مبني فوق أرض غير موجودة.

الأقوى نعملها مثل فريق هندسي:

[text]



Builder A يعمل الآن الشيء الموجود على critical path:

[text]

S1 BUILD

Builder B بنفس الوقت ما يلمس Production. نخليه يجهز الشيء التالي الذي لا يعتمد على مخرجات S1 النهائية، مثلاً:

[text]

S2/S3 DESIGN ATTACK

+

automation tooling design

+

test fixtures

+

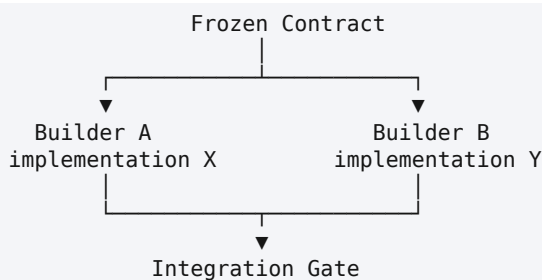
performance harness

هيك لما A يخلص S1 ونقبله، B يكون مجهز التصميم التالي ونوفر وقت حقيقي.

وفي طريقة أقوى لاحقاً؛ لما تكون المرحلة قابلة للتقسيم إلى وحدتين مستقلتين بعقد ثابت، نخلي الاثنين يبنوا بالتوازي على branches/artifacts منفصلة.

مثلاً:

[text]



لكن ممنوع نهائياً الاثنين يعدلوا نفس الملفات.

القواعد اللي رح تمنع الفوضى:

لكل Builder working copy/branch منفصلة، ونفس baseline hash بالبداية.

لكل واحد File Ownership مكتوب. تقاطع ملف واحد = ممنوع إلا بتفويض جديد.

Builder B لا يعدّل أي CLOSED أو artifacts قيد Audit عند A.

ما مندمج تقارير؛ مندمج artifacts بعد hashes/tests.

المرحلة التابعة لا تدخل Production قبل إغلاق مدخلها.

الأوديتور يراجع الأثر النهائي المدمج، وليس تقرير Builder منفصلين فقط.

إذا A و B اختلفوا في contract، ما بيحلوا الخلاف بينهم؛ STOP وبرجع القرار إلنا.

لهلق بالضبط، التوزيع الذكي:

[text]

BUILDER A

MUF S1 BUILD ONLY

الرسالة السابقة كاملة.

هو على critical path.

BUILDER B

لا S1 code

يشتغل بالتوازي على:

1. S2 + S3 dependency/design reconnaissance

2. جرد public APIs التي سيحتاجها S2

3. اكتشاف أين PolicyArtifact infrastructure ستحتاج ملفات جديدة

4. adversarial fixtures plan

5. تصميم unified automated research gate

6. اقتراح تقسيم ملفات يمنع التصادم مع S1

والسبب أن S2 نفسه يعتمد على S1 facts، فما بخله بيني S2 الآن. لكن يستطيع جهاز 80% من تصميمه بدون افتراض schema غير مغلقة.

وأنا أروح أبعد: ممكن نستفيد من Builder B كـ "Red-Team Builder" أحياناً بدل منفذ ثاني. يعني أثناء A بيني، B يأخذ التصميم ويحاول مسبقاً اكتشاف:

[text]

شو رح ينكسر بالكود؟

وين $O(n^2)$ المحتملة؟

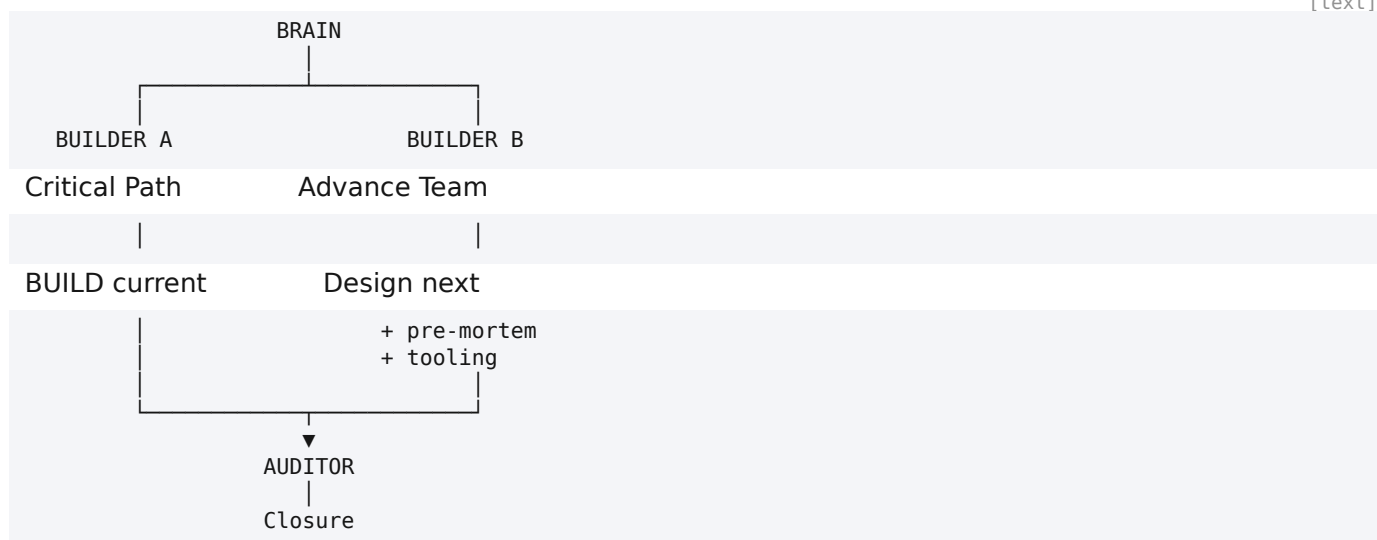
وين aliasing؟

وين future leakage؟

هذا بالضبط كان ممكن يصيد مشاكل S0 قبل ما توصل للأوديتور وبوفر جولات P1/P2.

فأفضل تنظيم لنا من هلق:

[text]



وهذا فعلياً ممكن يقص الوقت كثير بدون ما نقص الدقة.

فاي، افتح Builder ثاني. لا تعطيه شي من عندك حالياً. لما تفتحه، ابعتله الهاندأوف الأساسي، وبعدها أنا بعطيك مهمة مخصوصة لـ Builder B. بحيث يشتغل من اللحظة الأولى بالتوازي وما يخرب critical path.

طلب حزين شو لج اعمل بيلدر ثاني جنبو ونصير نعمل تتين تتين سوا شو رايك

gpt-5.5-instant

الحالة الرسمية: S0 CLOSED بالكامل، S1 صار DESIGN READY FOR BUILD، والـ 196/196 baseline و 1139/1139 و 36/36 Field Runner. والمشكلة التي أشرت لها حقيقية: إذا طبقنا نفس كثافة S0 على كل مرحلة، المشروع سيتباطأ أكثر مما يلزم. لكن الحل ليس تخفيف الدقة؛ الحل تغيير "وحدة العمل" وطريقة توزيع التدقيق.

S0 أخذ وقتاً استثنائياً لسبب: كان طبقة تأسيسية. خطأ واحد في immutability أو InformationKey ينتشر لكل شيء فوقه. لذلك كان يستحق الهجوم بالمجهر. لا يجب أن نعطي كل descriptor لاحقاً نفس وزن نواة الزمن والهوية.

من الآن أقترح نظام سرعة جديد مبني على المخاطر.

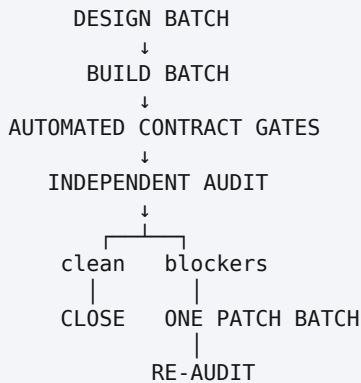
[text]

بدل:

```
Design
→ Review
→ Patch
→ Review
→ Build
→ Audit
→ Patch
→ Re-audit
→ Closure
→ Guard repin
→ Audit guard
→ ...
```

نشتغل:

[text]



والفرق الأساسي: نجمع المشاكل كلها في Patch واحد بدل قطّارة.

كمان نصنف المراحل حسب خطورتها:

RED: الزمن، السببية، execution/economics، TRAIN/OOS firewall، هذول ياخذوا التدقيق الثقيل.

GREEN: schemas وصفية، visualization، reports، adapters، AMBER: Turning Points، hierarchy، State Graph، estimands، تدقيق قوي لكن مو دوامة لا نهائية.

Build + automated gates مغلقة. audit + واحد.

S0 كان S1. RED أقرب لـ AMBER/GREEN لأنه مبني فوق S0 وممنوع فيه prediction أصلاً.

وفيه تسريع أكبر: ما عاد لازم نغلق كل ملف كأنه مشروع مستقل. نقدر نعمل "Research Milestone Batch" مع الحفاظ على حدود كل substage.

مثلاً بعد S1:

[text]

S1 Price Path



S2 Witness Adapter



S3 Policy Infrastructure

بدل 3 أسابيع من الإجراءات:

[text]

معاً S2 + S3 نصمم
لكن ننفذ:
S1 → gate
S2 → gate
S3 → gate

ثم Audit واحد للميلستون كله

مع hashes منفصلة لكل substage.

إذا S2 فشل، ما نعتبر S3 صالحاً فوّه. لكن ما نكرر كل الأعمال الإدارية ثلاث مرات.

الأهم من كل ذلك: نعمل Automation للحكمة التي كررناها يدوياً 20 مرة.

نحتاج لاحقاً أداة واحدة، مثلاً:

[text]

research_gate

تقوم آلياً بـ:

[text]

manifest verify

closed-file non-touch

authorized diff

private-import scan

forbidden-language/AST scan

dedicated tests

full tests

future-prefix tests

hash artifact

complexity gate

mutation battery

field runner guard

وتطلع:

[text]

GATE_REPORT.json

بدل ما البيلدر والأوديتور يعيدوا كتابة 500 سطر إثبات لكل جولة.

هون ممكن نريج وقت ضخم بدون تخفيف أي معيار.

كمان الـ Field Runner guard لا لازم نعمل له قصة مستقلة بعد كل Closure. اللي صار مرتين كشف flaw بالـ workflow نفسه:

[text]

كل MANIFEST رسمي جديد

↓

guard يصير stale

↓

Patch

↓

Audit

هذا تصميم تشغيلي غير قابل للتوسع.

الحل المستقبلي: الحارس الخارجي لا يثبت MANIFEST digest hard-coded داخل test source, بل يستهلك "accepted" "baseline seal" مستقل موقع/مهتم يُحدث فقط ضمن Closure. الحارس نفسه يبقى ثابتاً ويقول:

[text]

اقرأ baseline seal

↓

تحقق من هويته

↓

تحقق من MANIFEST

بدل:

[text]

"...expected_sha = "899feb

داخل الكود.

هيك ما منعدل test كل مرحلة. طبعاً هذا تغيير حوكمة، لذلك ما نعمله خلسة أثناء S1. نسجله كتحسين workflow ونبنيه بجولة مستقلة صغيرة بعد S1 أو بالتوازي إذا ما لمس المشروع.

وفي البحث نفسه، أكبر توفير للوقت سيكون أننا ما نبني كل شيء قبل ما نسأل هل في معلومة.

أنا لا أريد:

[text]

S1
S2
S3
S4

...

S12

سنة شغل

↓

وبعدين نكتشف no edge
بدنا Early Kill Gates.

الخريطة الأسرع:

[text]

S1 Price facts

↓

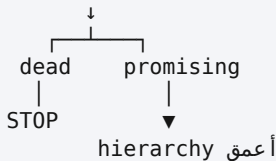
S2 Turning-point witness

↓

أول representation بسيطة شرعية

↓

EARLY INFORMATION SCREEN



لكن بدون فتح Final OOS أو الغش. نستخدم Development فقط.

يعني إذا بعد أول تمثيل قانوني وجدنا:

[text]

price-only baseline

≈

fancy representation

ما منبني 8 طبقات زينة.

وفي المقابل لو شطنا incremental information مستقرة على Development، نبرر الاستثمار بالطبقات الأغنى.

فأنا أقترح من الآن "Fast Research Protocol":

[text]

1. كل مرحلة:

design شامل.

2. Review:

blockers فقط.
style patches لا.

3. Build:

batch واحد.

4. Automated gates:

. كل الأشياء الميكانيكية

5. Auditor:

. يبحث فقط عن خرق قابل لإعادة الإنتاج

6. Patch:

مرة واحدة مجمعة قدر الإمكان.
. سببي جديد، طبعاً يتصلح blocker إذا بقي

7. Closure:

batch.

8. كل 2-3 مراحل:

. إداري مكرر audit شامل بدل milestone audit

9. Early Information Gates:

. نقل الفروع عديمة الفائدة بسرعة

10. Final OOS:

. يبقى مقفول مثل القبو

والشيء الذي لن أسرّعه مهما صار:

[text]

future leakage
TRAIN/OOS contamination
identity/availability
ACTUAL/PROXY
final evaluation
execution/economics

لأن خطأ فيها ممكن يعطيك نظاماً يبدو رابحاً وهو وهم.

أما:

[text]

اسم schema
ترتيب ملف
docstring
fixture قديم
visualization
تنسيق تقرير
ما رح أخليك تنتظر بسببه.

وبالنسبة لمكاننا الحالي تحديداً: ما منوقف S1 لنصمم نظام الأتمتة الآن. هذا بحد ذاته رح يصير تأخير جديد. S1 صار جاهز BUILD، فنعطيه للبيدر كما في رسالتي السابقة ونخله يشتغل.

ومن S2 أطبق البروتوكول الأسرع: تصميم S2/S3 كميلستون مترابط من البداية، automated gate موحد، audit مجمع، وEarly Information Gate بأقرب نقطة علمياً مسموحة.

هيك السرعة ما بتيجي من "طنش الاختبارات". بتيجي من إزالة الشغل الإداري المكرر، وتجميع القرارات، وقتل الأفكار الضعيفة بكير.

هلق هيك كل خطوة بدنا نضل فيها يومين تلاته ؟ مايبخلص المشروع لبعد سنه شو الحل بدون مانفقد الدقه

gpt-5.5-instant

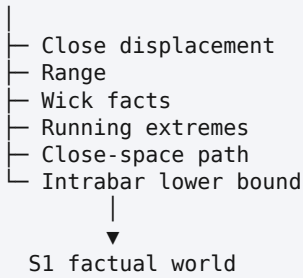
الحالة الرسمية: S1 DESIGN صار جاهز للبناء عندي. راجعت RC1 تحديداً مقابل البلوكرين وباقي H1 المجمع، والاثنان انغلقوا بدون خلق اعتماد دائري جديد. حكمي: DESIGN READY FOR BUILD. ما رح أفتح Design Patch ثالث.

لكن قبل ما أعطيك رسالة البيدر، بدّي أوضح شو حسمنا لأن هلق فعلاً الصورة صارت نظيفة:

[text]



Price Facts



ولا يوجد:

Turning Point

Wave

Trend

Importance

Prediction

والgap صار مضبوط:

[text]

10:00

10:01

10:03

بدون دليل اكتمال:

"10:02 لم أرها بعد"



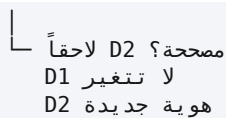
Missing Fact لا

sealed declared-grid coverage: مع

"10:02" غير موجودة ضمن D1



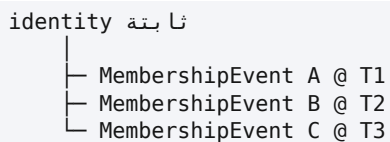
EXPECTED_GRID_KEY_NOT_OBSERVED



والEpisode صار:

[text]

Episode E



ما عاد عضو مستقبلي يعيد كتابة الحلقة.

الآن BUILD S1 مصرح ضمن الضوء الأخضر السابق، ولكن فقط S1.

ابعث للبيدر:

[text]

MUF V1 — S1 PRICE PATH PRIMITIVES

BUILD ONLY

OWNER AUTHORIZATION:
BUILD S1 ONLY.

S0 is CLOSED.

S1 design readiness:
ACCEPTED FOR BUILD.

Governing S1 design authority:

RC1
→ H1
→ S1 FINAL IMPLEMENTATION DESIGN

where RC1/H1 supersede only their explicit scopes.

Official baseline:

MANIFEST SHA256:
899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84

MANIFEST:
196/196 OK

Project:
1139/1139

Field Runner:
36/36

MUF S0:
CLOSED

No S2+ authorized.

ALLOWED NEW FILES ONLY

Production:

src/trading_system/market_understanding/price_path.py
src/trading_system/market_understanding/path_schemas.py

Tests:

tests/test_muf_s1_price_path.py
tests/test_muf_s1_path_schemas.py

S1 must be additive only.

If an existing file appears necessary to modify:
STOP — OWNER REVIEW.

Do NOT modify S0 __init__.py merely for exports.
Public import may use module paths directly until closure/authorized patch.

ABSOLUTELY FORBIDDEN

No modification to existing src/tests.

No MANIFEST update.

No docs.

No field_runner.
No S0 changes.
No S2.
No swing detector execution.
No turning-point detection.
No wave/run/segment ontology.
No hierarchy.

No $\alpha/\beta/\gamma/\delta$.

No policy fit.

No calibration.

No quantile/threshold/fixed hidden window.

No Model/Strategy/Signal/PnL.

No OOS/final evaluation.

No narrative population.

No statistical independence claim.

1 — PRICE INPUT CONTRACT

Implement S1 price semantics exactly as accepted:

PUBLISHED_OHLC_FACT

Price fields must NOT use ACTUAL/PROXY semantics.

Required provenance:

source_identity

dataset_identity

timeline/axis binding

published bar record reference

Validate legal OHLC:

finite prices;

source-lawful price domain;

$H \geq \max(0, C);$

$L \leq \min(0, C);$

$H \geq L.$

No clipping/repair/coercion.

No tick-path reconstruction claim.

Volume/flow not required.

2 — LAWFUL STREAM / ADJACENCY

Implement the accepted strict causal stream contract.

For one accepted stream:

new key must be lawfully strictly greater than previous accepted key.

Reject:

duplicate key:

S1_DUPLICATE_OBSERVATION_KEY

out-of-order comparable key:

S1_OUT_OF_ORDER_OBSERVATION

incomparable:

fail closed under S0 error contract

cross timeline/axis:

reject.

Never silently sort input.

OBSERVATION_ADJACENT =

consecutive accepted observations under this contract.

It is NOT automatically GRID_CONTIGUOUS.

Same-batch chronology remains unproven where S0 says so.

3 — PRICE PATH PRIMITIVES

Implement factual primitives only:

CLOSE_DISPLACEMENT

CLOSE_PATH_STEP

BAR_RANGE

OPEN_CLOSE_DISPLACEMENT

UPPER_WICK

LOWER_WICK

RUNNING_HIGH_SO_FAR

RUNNING_LOW_SO_FAR

Direction:

UP / DOWN / FLAT

for observed close displacement only.

Forbidden public semantics:

trend

bullish

bearish

structure

impulse

correction.

No turning-point or wave vocabulary except explicit schema guard tests proving it is absent.

4 — INTRABAR AMBIGUITY

Never infer H-before-L or L-before-H from OHLC.

INTRABAR_PATH_LENGTH_EXACT:

UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY)

INTRABAR_PATH_LENGTH_LOWER_BOUND:
BOUND

using the accepted discrete total-variation formula:

(H-L)

+

min(

$$\begin{array}{l} |0-H| + |L-C|, \\ |0-L| + |H-C| \end{array}$$

)

bound_basis:

DISCRETE_TV_TRIANGLE_INEQUALITY

No continuity claim.

No claim intermediate prices traded.

Upper bound:

UNAVAILABLE.

5 — RUNNING EXTREME TIES

Tie origins are an UNORDERED SEMANTIC SET.

No winner.

No first true extreme.

No tie-break.

Any canonical storage ordering is non-semantic only.

Same legal tie-set:

same semantic/canonical identity independent of insertion representation.

deterministic_sequence must not establish semantic winner.

6 — GRID / OBSERVATION MISSINGNESS

Do NOT emit a factual absence merely because the stream skipped an expected key.

Reuse the existing PUBLIC CLOSED Stage 4C-1 coverage/completeness contract.

ExpectedGridKeyNotObservedRecord may be emitted ONLY if:

- declared grid exists;
- sealed observation domain exists;
- expected key is inside covered domain;
- timestamp-set comparison is available;
- coverage evidence establishes GRID_OBSERVATIONS_MISSING;
- claim is scoped to that sealed dataset/source/timeline/cadence domain.

Without lawful authority:

NO factual absence record.

Return/reject according to accepted RC1:

S1_ABSENCE_REQUIRES_CLOSED_DOMAIN

and existing typed unavailable/not-configured/unknown state.

Never claim:

market/exchange did not produce a bar.

Never forward fill.

Never synthesize a bar.

GRID_CONTIGUOUS is separate from OBSERVATION_ADJACENT.

7 — AXIS / TIME

POSITIONAL:

no invented timestamp.

TIME_INDEXED:

lawful timezone-aware time required.

Bar-count facts and wall-clock duration must remain separately typed.

No silent axis conversion.

8 — SAME INFORMATION BATCH

Reuse S0 InformationBatchKey.

Mechanical processing order is not market chronology.

If a requested primitive/transition requires chronology unproven inside a batch:

UNDEFINED(SAME_BATCH_ORDER_UNPROVEN)

or deterministic contract rejection as designed.

Never use deterministic_sequence to create market chronology.

9 — DESCRIPTOR SPECS

Implement the accepted S1 DescriptorSpec infrastructure.

Allowed factual running descriptors:

- observed_close_path_length
- grid_contiguous_close_path_length
- running_high_so_far
- running_low_so_far
- bar_count_observed
- bar_count_grid
- sum_close_displacement

Each declares:

identity/version

stage

required inputs

availability rule

boundary contract

missingness

denominator semantics

incremental state

policy dependencies

At S1:
policy dependencies must be none.
No final-wave descriptors.
No future-normalized values.
Zero denominator:
typed UNDEFINED.
No epsilon.
No inf.

10 — SCHEMA-ONLY FOUNDATIONS

Implement schemas only, not market population logic:

CausalEpisodeRecord
EpisodeMembershipEvent
MarketStateTransitionRecord
ExplanationRecord

Episode:
anchor identity separate from membership.

Membership:
append-only event.
Future member does not change episode_id.

No statistical independence claim.
RESEARCH-DEBT-024 remains OPEN.

Transition:
literal factual state/descriptor delta only.
No invented regime threshold.

Explanation allowed states only:

MONITORING
PATTERN_REQUIREMENTS_SATISFIED
CONTRADICTED
SUPERSEDED

No probability.
No weights.
No support score.
No parallel narrative engine.

11 — FUTURE APPEND INVARIANCE

For every S1 fact emitted at $\leq T$:

append legal future observations.

Previously emitted facts must remain byte/semantically identical.

Mandatory for:

primitives
bar derivatives
running-extreme snapshots
descriptors

absence facts scoped to sealed dataset identity
schema-only episode/membership/transition/explanation records.

Evolution:

new record/event only.

No repaint.

12 — PERFORMANCE

Streaming/incremental.

Target:

$O(1)$ amortized per new bar

$O(n)$ total.

No full-history rescan per bar.

Running-extreme tie-set:

$O(k)$ memory where k is current number of tied origins;

no arbitrary truncation.

No $O(n^2)$.

Use deterministic operation-count/scaling tests,
not wall-clock only.

13 — MANDATORY TESTS

Implement the full accepted adversarial matrix from:

S1 FINAL DESIGN

+

H1

+

RC1.

At minimum include all original attacks 1-20 plus H1 A-G and RC1 A1-A3/B1-B3.

Critical mutation/adversarial gates must include:

A) OHLC chronology assumption introduced

→ test fails.

B) path exactness falsely promoted from BOUND/UNAVAILABLE

→ fails.

C) silent row sorting/out-of-order acceptance

→ fails.

D) duplicate key creates primitive

→ fails.

E) tie-set winner introduced

→ fails.

F) missing observation emitted without closed-domain authority

→ fails.

G) "market bar absent" semantics introduced

→ schema/wording guard fails.

H) price ACTUAL/PROXY label introduced
→ fails.

I) DisplacementRun/segment ontology introduced
→ AST/design guard fails.

J) Episode membership added into anchor record
→ fails.

K) future member changes episode identity
→ fails.

L) same-batch deterministic_sequence creates chronology
→ fails.

M) fixed hidden window introduced
→ fails.

N) history rescan creates $O(n^2)$
→ scaling gate fails.

O) future append changes old S1 fact
→ fails.

Tests must be non-vacuous.

14 — MUTATION PROOFS

On isolated copies prove at minimum:

1) force H-before-L traversal:
chronology/path test fails.

2) silently sort out-of-order input:
adjacency test fails.

3) resolve tie by first origin:
tie permutation/winner test fails.

4) allow absence from stream skip without coverage:
absence authority test fails.

5) add membership_refs into episode anchor:
episode schema test fails.

6) restore history scan:
complexity gate fails.

No residue.

15 — PUBLIC API / PRIVATE IMPORT GATE

Reuse only PUBLIC CLOSED/SO APIs.

No private imports.

Before implementation inventory exact reused symbols.

If required Stage 4C-1 completeness functionality exists only privately:
STOP — OWNER REVIEW.

Do NOT copy/reimplement its private semantics.

16 — NON-TOUCH

Before build snapshot:

all existing src/tests

docs

MANIFEST

field_runner.

After build:

every pre-existing file byte-identical.

Only four new S1 files authorized.

MANIFEST remains:

899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84

196/196 existing entries OK.

S1 new files remain unmanifested pending audit/closure.

17 — TEST GATES

Run:

S1 dedicated tests.

S0 dedicated:

67/67.

Full project:

baseline 1139 + actual S1 tests.

Field runner:

36/36.

Report actual counts.

Do not guess.

18 — DELIVERY

Return:

MUF_S1_BUILD_REPORT

Status exactly:

IMPLEMENTED — PENDING AUDIT

or

STOP — OWNER REVIEW

Include:

- exact new files
- SHA256 all new files
- zero modified pre-existing files proof
- public APIs reused
- absence-authority public contract actually used

- primitive/schema inventory
- metric semantics inventory:

EXACT / BOUND / UNAVAILABLE

- complexity evidence
- dedicated/S0/full/field_runner tests
- mutation proofs
- MANIFEST before/after identity
- any design contract not representable exactly

No MANIFEST update.

No docs closure.

No S2.

.STOP

والآن صرنا فعلياً بمرحلة أكثر عملية. بعد S1 إذا اشتغل صح، النظام سيقدر يأخذ شارت BTC ويقول لك بدقة:

[text]

هون تحرك الإغلاق X+

هون Y-

هون Range كذا

هون عندي Running High مش Turning Point

هون High/Low order مجهول

هون في observation gap بالداتا وليس "gap بالسوق"

هون عندي tie بين أكثر من extreme

يعني عم نبني «العين» قبل «التفسير».

الخطوة المطلوبة الآن: هذا النص لليلدر. بعد ما يرجع MUF_S1_BUILD_REPORT، ما رح نفتح نقاش تصميم جديد إلا إذا التنفيذ نفسه أثبت خرقاً حقيقياً. وقتها Audit تنفيذي شامل مرة واحدة.

MUF V1 S1 — READINESS CORRECTION RC1

TWO BLOCKERS ONLY — DESIGN ONLY

References: MUF V1 S1 FINAL IMPLEMENTATION DESIGN

(deliverables/53_...) + S1 FINAL DESIGN HARDENING H1 (deliverables/54_...).

Scope: exactly two blockers. Everything else in H1 is FROZEN for readiness review (price semantics, adjacency, tie-set, DisplacementRun deferral, intrabar lower-bound proof, direction, path metrics, DescriptorSpec, Transition schema, Explanation schema, performance design, certification boundary — all unchanged).

No code. No tests. No MANIFEST. No S2.

Status: DESIGN ONLY — READINESS REVIEW

RC1-1 — ABSENCE REQUIRES OBSERVATION-DOMAIN AUTHORITY

RC1-1.0 Finding — does a lawful existing completeness authority exist?

YES — and it is reused, not invented. The repo already contains a PUBLIC

CLOSED completeness/coverage contract with exactly the required shape: the

Stage 4C-1 declared-grid coverage contract over sealed observations

(research/trajectory/trajectory_stage4c.py, CLOSED; certified in

docs/FINAL_VALIDATION.md "6.2A-4 Stage 4C-1 closure boundary"):

Existing contract element Exact content

coverage status vocabulary GRID_OBSERVATIONS_COMPLETE / GRID_OBSERVATIONS_MISSING /

OFF_GRID_OBSERVATIONS_PRESENT / GRID_OBSERVATIONS_DEFECT_BOTH /

GRID_COMPLETENESS_UNKNOWN / COVERAGE_UNAVAILABLE (UNAVAILABLE)

decision rule "completeness decided by timestamp-SET comparison against the declared grid (never counts alone)"; without a declared grid coverage is GRID_COMPLETENESS_UNKNOWN

claim scope COVERAGE/CLAIM_SCOPE =

OBSERVATIONS_VS_DECLARED_GRID_WITHIN_SEALED_DATA_ONLY

explicit non-claims COVERAGE/NO_CLAIM = zzz.txt — سجل المحادثات

NO_FEED_COMPLETENESS_NO_MARKET_COMPLETENESS_NO_CALENDAR

domain closure observations are sealed (immutable sealed timeline/observation truth); later corrected artifacts are new seals, never rewrites

supporting rule causal_hrf.py: "coverage completeness is unknown without a cadence contract"

Typed result vocabulary is exactly the existing TypedState members

(NOT_CONFIGURED, UNAVAILABLE, UNDEFINED, NOT_APPLICABLE) plus the

coverage tokens above — no new typed state is invented. The order's

"UNKNOWN / UNAVAILABLE / NOT_CONFIGURED" maps to the existing contract as:

GRID_COMPLETENESS_UNKNOWN (coverage token) / TypedState.UNAVAILABLE /

TypedState.NOT_CONFIGURED respectively.

RC1-1.1 Conceptual authority → existing contracts (no new semantics)

ObservationDomainClosureRef (conceptual) is realized by composition of

existing CLOSED contracts — it introduces no new completeness semantics:

Conceptual field Existing contract anchor

source_identity S0 Schemalidentity of the sealed observation source

dataset_identity sealed observation-set identity (S0 record/schema identity of the sealed dataset seal)

timeline_id S0 InformationKey timeline

cadence_contract_identity cadence contract (declared-grid basis; published kline cadence where integrated)

covered_domain the DECLARED timestamp grid span (exact keys)

closure/completeness evidence timestamp-SET comparison result of sealed observations vs the

declared grid (GRID_OBSERVATIONS_COMPLETE / per-key GRID_OBSERVATIONS_MISSING;

GRID_COMPLETENESS_UNKNOWN means no authority)

availability_information_key the InformationKey at which the sealed coverage comparison is established (S0 availability rules)

RC1-1.2 Corrected absence contract (emission gate)

Invariant of meaning: NOT_YET_OBSERVED ≠ PROVEN_NOT_OBSERVED_WITHIN_CLOSED_DOMAIN.

Merely not seeing key K in the accepted stream is never sufficient.

ExpectedGridKeyNotObservedRecord may be emitted ONLY under case B.

Case A — stream prefix advanced beyond K, but no closed-domain authority.

(No declared grid, or no sealed coverage comparison, or coverage status

GRID_COMPLETENESS_UNKNOWN / COVERAGE_UNAVAILABLE.)

Absence claim MUST NOT be emitted — no factual missing record exists.

Deterministic outcome: emission rejected with

SchemaViolation("S1_ABSENCE_REQUIRES_CLOSED_DOMAIN"); the gap determination

returns the existing typed states — TypedState.UNAVAILABLE (cannot decide)

/ TypedState.NOT_CONFIGURED (absence capability unconfigured in the given

domain) / coverage status GRID_COMPLETENESS_UNKNOWN — never a fact.

Case B — legally closed observation domain.

A declared grid covers K; the sealed observation domain is coverage-compared

against that grid (timestamp-set comparison) by the availability key; the

comparison establishes GRID_OBSERVATIONS_MISSING at K within sealed data

only.

Only then EXPECTED_GRID_KEY_NOT_OBSERVED may be published, carrying the

composed domain_closure_ref (RC1-1.1) as required provenance, and its claim

remains scoped: K is not observed within the closed sealed domain D — never

"the market/exchange did not produce a bar" (H1-6 scope stands), never feed

completeness, never market completeness.

Critical invariant — future arrival never rewrites published absence.

Absence facts are published only where the observation-domain contract makes a

future arrival of K incompatible with the same closed domain: the sealed

zzz.txt — سجل المعادلات

dataset D is immutable. A later corrected artifact containing K is a new dataset D2 with distinct dataset/provenance identity (a new seal); D1's published absence remains untouched domain-scoped history; corrections flow through the higher-authority supersession/correction-event model — never by rewriting history.

RC1-2 — EPISODE IDENTITY ≠ MEMBERSHIP

CausalEpisodeRecord.membership_refs[] is REMOVED. It conflicts with the accepted higher-authority MUF design (D1/D2):

[text]

EpisodeAnchorIdentity ≠ EpisodeMembershipEvent (membership may become known later)

RC1-2.1 Corrected CausalEpisodeRecord — anchor/creation facts ONLY

Field Semantic Identity?

episode_identity S0 compute_record_identity over the identity fields below identity

schema_identity S0 schema hash (MUF_S1_EPISODE_ANCHOR, V1) identity

timeline_id / axis key binding identity

anchor_information_key creation/anchor key of the episode identity

anchor_rule_version the anchor/creation rule version that defines the episode identity

anchor_fact_ref CausalRecordReference to the anchoring fact non-identity

provenance creation provenance non-identity

availability_information_key earliest key at which the anchor is established non-identity

No membership field exists on this record (by schema: frozen payload, closed field set — unknown fields fail closed).

RC1-2.2 New schema: EpisodeMembershipEvent (append-only)

text

EpisodeMembershipEvent {

episode_id, // references CausalEpisodeRecord.episode_identity

member_fact_ref, // CausalRecordReference to the member fact

membership_information_key, // earliest lawful InformationKey at which
// this membership is known

provenance,

event_identity, // S0 compute_event_identity

schema_identity // MUF_S1_EPISODE_MEMBERSHIP, V1

}

Membership is append-only (S0 EventKind/EventRecord +

AppendOnlyEventLedger discipline). If correction/removal of membership is ever required: the higher-authority supersession/correction-event model applies — historical membership events are never mutated.

RC1-2.3 Requirements (binding)

I-EP-S1-1: adding a member never changes episode_id.

I-EP-S1-2: a published CausalEpisodeRecord is never mutated to add a member (membership lives only in EpisodeMembershipEvents).

I-EP-S1-3: a membership event's availability is the earliest lawful InformationKey at which that membership is known.

I-EP-S1-4: episode accounting makes NO statistical independence claim; RESEARCH-DEBT-024 remains OPEN.

I-EP-S1-5: S1 only defines these schemas; it does NOT populate market episodes.

EXACT SCHEMA DELTAS

Element Delta

ExpectedGridKeyNotObservedRecord (H1-6) + required domain_closure_ref (composed per RC1-1.1: source/dataset identity, timeline, cadence contract, covered declared grid, sealed coverage-comparison evidence, availability key); emission gated on case B only (RC1-1.2); per-key

reason EXPECTED_GRID_KEY_NOT_OBSERVED now cites coverage evidence
GRID_OBSERVATIONS_MISSING within sealed data only
H1-6 typed gap vocabulary GRID_OBSERVATION_MISSING (singular, invented) retired → the existing
Stage 4C token GRID_OBSERVATIONS_MISSING as evidence status + existing TypedState members for
typed results; GRID_COMPLETENESS_UNKNOWN / COVERAGE_UNAVAILABLE govern case A
error/reason vocabulary + S1_ABSENCE_REQUIRES_CLOSED_DOMAIN (case A rejection); no other error
change
CausalEpisodeRecord (15.6/§12.1) membership_refs REMOVED; fields = anchor/creation facts only
(RC1-2.1); schema family MUF_S1_EPISODE_ANCHOR
EpisodeMembershipEvent NEW schema (RC1-2.2), append-only, schema family
MUF_S1_EPISODE_MEMBERSHIP
MarketStateTransitionRecord / ExplanationRecord UNCHANGED (frozen)
attack/test & build-plan descriptions per the two sections below
ATTACKS A1-A3 / B1-B3

Attack Violated contract Minimal fixture Expected rejection / state Design gate
A1 absence claimed from prefix stream without authority RC1-1.2 case A observed 10:00, 10:01, 10:03
in accepted stream; no declared-grid/ sealed coverage authority; attempt absence record for 10:02
reject SchemaViolation("S1_ABSENCE_REQUIRES_CLOSED_DOMAIN"); determination = UNAVAILABLE /
NOT_CONFIGURED / GRID_COMPLETENESS_UNKNOWN; no factual missing record emission gate on
domain_closure_ref presence + coverage status
A2 lawful closed-domain authority covering 10:02; 10:02 absent RC1-1.2 case B same times, with
declared grid covering 10:02 and sealed-domain coverage comparison establishing
GRID_OBSERVATIONS_MISSING at 10:02 factual EXPECTED_GRID_KEY_NOT_OBSERVED allowed, scoped
to the sealed domain D only schema requires domain_closure_ref; claim-scope lock ("within sealed
data only")
A3 later corrected dataset rewrites history RC1-1.2 invariant absence published under closed dataset
D1; later corrected dataset D2 contains 10:02 D1 history not rewritten; D2 has distinct
dataset/provenance identity (new seal); correction via supersession/correction-event model only
immutability + dataset-identity distinctness test
B1 membership mutation via later member RC1-2 I-EP-S1-1/2 create episode E; append member A;
append member B later same episode_id; two append-only EpisodeMembershipEvents;
CausalEpisodeRecord bytes unchanged identity-anchor test + append-only ledger discipline
B2 mutate CausalEpisodeRecord.membership_refs RC1-2.1 attempt to set/read membership_refs field
does not exist / schema reject (SchemaViolation on unknown field) closed frozen-payload schema test
B3 future member changes historical episode identity RC1-2 I-EP-S1-1/3 membership known after
episode publication impossible/reject: identity fields exclude membership; membership availability =
earliest lawful key only identity-field whitelist + availability rule (I-EP-S1-3)

BUILD-PLAN DELTA ONLY

File Delta

path_schemas.py ExpectedGridKeyNotObservedRecord gains required domain_closure_ref +
case-B-only emission gate; gap vocabulary aligned to existing Stage 4C tokens; CausalEpisodeRecord
reduced to anchor fields (schema family MUF_S1_EPISODE_ANCHOR); new EpisodeMembershipEvent
schema (family MUF_S1_EPISODE_MEMBERSHIP); + S1_ABSENCE_REQUIRES_CLOSED_DOMAIN
rejection code
price_path.py no change (gap detection remains a stream observation only — never an absence claim;
detection state cannot bypass the RC1-1.2 gate)
tests/test_muf_s1_path_schemas.py + attacks A1-A3 (absence authority battery), B1-B3 (episode
identity ≠ membership battery)
tests/test_muf_s1_price_path.py no change (frozen scope)
Complexity, performance design, and public-API reuse are unchanged. S0 remains
untouched. Nothing outside the two blockers was modified.

OPEN / NOT_CONFIGURED (delta to H1 list)

Absence feature (ExpectedGridKeyNotObservedRecord): CONFIGURABLE in S1 V1

سجل المصادقات — ZZZ.LXX

under case B only, because a lawful completeness authority exists (Stage 4C-1 sealed-observations-vs-declared-grid coverage contract — reused, not invented). In domains lacking a declared grid / sealed coverage comparison, the capability is NOT_CONFIGURED / UNAVAILABLE (case A) and no absence fact is ever produced there.

Episode schemas: schema-only (I-EP-S1-5) — population of market episodes stays NOT_CONFIGURED at S1.

All other OPEN/NOT_CONFIGURED items from H1 remain exactly as stated (DisplacementRun deferred; RESEARCH-DEBT-024 OPEN; waves/turning points/hierarchy/ α - β - γ - δ /policies/regime/narrative/edge/Model/Strategy/Signal/PnL out of scope).

STATUS

DESIGN ONLY — READINESS REVIEW

Two blockers corrected: absence claims now require (and reuse) an existing CLOSED observation-domain authority with a no-rewrite invariant, and episode identity is separated from append-only membership. No code. STOP.

gpt-5.5-instant

الحالة الرسمية: H1 أصلح الستة اللي رجعناهم فعلاً، وما رح أرجعه لتجميل أو أسماء. لكن بعد Cross-check مع الدستور الأعلى D1/D2، التصميم لسا مو جاهز BUILD. عندي بلوكران فقط، وهالمرة واحد منهم موجود تحديداً لأن H1 حسّن معنى gap لكنه ما حسم «متى نملك حق إعلان الغياب»، والثاني تعارض مباشر مع تصميم الـ CausalEpisode append-only المقبول سابقاً.

يعني الحكم:

[text]

S1 H1	
— Price source semantics	✓
— Lawful adjacency	✓
— Extreme ties	✓
— DisplacementRun deferred	✓
— Discrete intrabar LB	✓
— Missing observation semantics	✓
— Authority to PROVE missing	● BLOCKER
— Episode membership model	● BLOCKER

BUILD = NO
أخطر شيء بقي: كيف عرفنا أن الـ Grid key «مفقود» أصلاً؟
H1 صار يقول بشكل صحيح:

[text]

EXPECTED_GRID_KEY_NOT_OBSERVED
بدل:

[text]

MARKET BAR DID NOT EXIST
ممتاز.

بس تخيل stream حي:

[text]

وصل:

10:00

10:01

10:03

هل بمجرد وصول 10:03 نقدر نقول:

[text]

NOT OBSERVED 10:02

ليس دائماً.

ممکن يكون:

[text]

delayed 10:02

أو:

[text]

source delivers out of order

أو:

[text]

dataset incomplete

أو الـ 10:02 موجود لكنه لم يُقرأ بعد.

H1 كتب:

[text]

detection_key =

lawful InformationKey at which non-observation

within dataset domain is proven

لكن هذا يعيد صياغة السؤال بدل الإجابة عنه:

«مين أثبت إنه proven؟»

نحتاج authority واضحة مثل:

[text]

DatasetObservationDomain / Watermark / Closed interval

تقول مثلاً:

[text]

هذا المصدر/الملف يشهد أن كل expected keys

ضمن [A,B] أصبحت قابلة للفحص حتى cutoff X.

وقتها فقط:

[text]

expected key K

+

closed observation domain داخل K

+

scan/watermark تعداه قانونياً

+

K غير موجود

↓

EXPECTED_GRID_KEY_NOT_OBSERVED

بدونها:

[text]

NOT YET OBSERVED

≠

MISSING OBSERVATION

وهذا نفس مبدأ $\text{Origin} \neq \text{Availability}$ ولكن للغياب.

إذا ما في عندنا public CLOSED contract يثبت completeness/watermark، الجواب الصحيح:

[text]

NOT_CONFIGURED

ولا نخترع واحد.

هذا بلوكر سببي حقيقي.

CausalEpisodeRecord رجع خطوة للخلف عن تصميم MUF المقبول.
H1 ورث من التصميم الأساسي:

[text]

CausalEpisodeRecord {

```
...  
membership_refs[]
```

{

لكن D1/D2 اللي فوق S1 بالسلطة حسموا سابقاً:

[text]

EpisodeAnchorIdentity

≠

EpisodeMembershipEvent

لأن أعضاء الـ episode ممكن يظهروا لاحقاً.

مثال:

[text]

T1:

Episode E created

members = [FVG]

T2:

BOS becomes available

T3:

flow fact becomes available

إذا membership_refs[] جزء من EpisodeRecord المنشور، أمامك خياران سيئين:

[text]

تعديل E

→ خرق immutability

أو

إنشاء Episode جديد

→ الهوية تتغير لأن عضو جديد ظهر

بينما التصميم الأعلى حسم:

[text]

Episode identity anchored once

ثم:

MembershipEvent(FVG)

MembershipEvent(BOS)

MembershipEvent(flow)

كل واحد عند InformationKey الخاص فيه.

إذن S1 schema لازم يكون فقط foundation مطابق لهذا، مش schema منافس.

الجزء الحلو: هدول ما بيحتاجوا H2 ضخم. ما رح نعيد Design كله. أعطيه Correction صغيرة جداً، نقطتين فقط. بعدها إذا سدّهم كما يجب أعطي BUILD مباشرة، بدون جولة أفكار جديدة.

ابعت للبيادر:

[text]

MUF V1 S1 — READINESS CORRECTION RC1

TWO BLOCKERS ONLY — DESIGN ONLY

Reference:

MUF V1 S1 FINAL IMPLEMENTATION DESIGN

سجل المحادثات — zzz.txt

+

S1 FINAL DESIGN HARDENING H1

Status:

DESIGN ONLY.

This correction is limited to TWO blockers only.

Everything else in H1 is FROZEN for readiness review.

No code.

No tests.

No MANIFEST.

No S2.

RC1-1 — ABSENCE REQUIRES OBSERVATION-DOMAIN AUTHORITY

Problem:

EXPECTED_GRID_KEY_NOT_OBSERVED is a factual claim of absence within an observation domain.

Merely not seeing key K yet is not sufficient.

NOT_YET_OBSERVED

≠

PROVEN_NOT_OBSERVED_WITHIN_CLOSED_DOMAIN.

Define the exact authority required before

ExpectedGridKeyNotObservedRecord may be emitted.

The design must reuse an existing PUBLIC CLOSED completeness/cadence/dataset contract if one exists.

Do NOT invent source completeness.

At minimum distinguish:

A)

stream prefix has advanced beyond K,
but source/dataset completeness is not proven.

Result:

absence claim must NOT be emitted.

Use explicit UNKNOWN / UNAVAILABLE / NOT_CONFIGURED according to the existing typed contract.

B)

a legally closed observation domain / completeness witness / watermark
proves that expected key K belongs to the inspected domain and the domain
is complete enough to decide its presence.

K absent.

Only then:

EXPECTED_GRID_KEY_NOT_OBSERVED may be emitted.

Required conceptual authority:

ObservationDomainClosureRef {

```
    source_identity,  
    dataset_identity,  
    timeline_id,  
    cadence_contract_identity,
```

```
covered_domain,  
closure/completeness_evidence,  
availability_information_key  
}
```

This is a conceptual contract only.

Use the actual existing public contract if one already exists.

If NO existing lawful completeness authority exists:

S1 V1 must mark this capability NOT_CONFIGURED rather than invent one.

Critical invariant:

future legal arrival of a previously "missing" observation must never
rewrite an already published absence fact.

Therefore an absence fact may only be published where its observation-domain
contract makes that future arrival incompatible with the same closed domain
(or requires a new corrected/superseding source artifact rather than rewriting
history).

Add attacks:

A1)

10:00, 10:01, 10:03 observed, no completeness/watermark authority.

Attempt absence record for 10:02.

→ reject / NOT_CONFIGURED / UNAVAILABLE.

No factual missing record.

A2)

same fixture with lawful closed-domain completeness authority covering 10:02.

10:02 absent.

→ factual EXPECTED_GRID_KEY_NOT_OBSERVED allowed.

A3)

absence record exists under closed dataset D1;

later corrected dataset D2 contains 10:02.

→ D1 history not rewritten.

D2 has distinct dataset/provenance identity.

Do not assume live-stream advancement alone proves absence.

RC1-2 — EPISODE IDENTITY ≠ MEMBERSHIP

Problem:

S1 currently proposes CausalEpisodeRecord.membership_refs[].

This conflicts with accepted higher-authority MUF design:

EpisodeAnchorIdentity

≠

EpisodeMembershipEvent

because membership may become known later.

Correct S1 schema-only foundation to match D1/D2.

CausalEpisodeRecord must contain only immutable episode anchor/creation facts
that define the episode identity.

Membership is append-only through a separate schema:

EpisodeMembershipEvent {

```
    episode_id,  
    member_fact_ref,  
    membership_information_key,  
    provenance,  
    event_identity/schema_identity
```

}

Requirements:

I-EP-S1-1:

adding a member never changes episode_id.

I-EP-S1-2:

published CausalEpisodeRecord is never mutated to add a member.

I-EP-S1-3:

membership event availability is the earliest lawful InformationKey at which that membership is known.

I-EP-S1-4:

episode accounting still makes NO statistical independence claim.

RESEARCH-DEBT-024 remains OPEN.

I-EP-S1-5:

S1 only defines these schemas.

It does NOT populate market episodes.

If correction/removal of membership is ever required:

use the higher-authority supersession/correction-event model;

never mutate historical membership.

Add attacks:

B1)

create episode E.

append member A.

append member B later.

→ same episode_id, two append-only membership events.

B2)

try to mutate CausalEpisodeRecord.membership_refs.

→ field must not exist / schema reject.

B3)

future member changes historical episode identity.

→ impossible/reject.

IMPACTED DELTAS ONLY

Update only:

- gap/absence schema semantics from H1-6;
- ExpectedGridKeyNotObservedRecord prerequisites;
- CausalEpisodeRecord schema;
- add EpisodeMembershipEvent schema;
- affected adversarial tests/build-plan descriptions.

Do NOT change:

price semantics
adjacency
tie-set
DisplacementRun deferral
intrabar lower-bound proof
direction
path metrics
DescriptorSpec
Transition schema
Explanation schema
performance design
certification boundary

DELIVERY

Return:

MUF-V1-S1-READINESS-CORRECTION-RC1

Status:

DESIGN ONLY — READINESS REVIEW

Include:

- corrected two contracts
- exact schema deltas
- attacks A1-A3 / B1-B3
- whether a lawful existing completeness authority actually exists

or the absence feature remains NOT_CONFIGURED

- build-plan delta only.

No code.

.STOP

وفي جواب لسؤال ممكن يخطر ببالك: «معقول كل ما نقصت شمعة بدنا كل هالفلسفة؟»

المشكلة مو الشمعة. المشكلة بالفرق بين:

[text]

ما شفتها

و:

[text]

ثبت أنها غير موجودة ضمن البيانات التي ادّعى اكتمالها
مثلاً لو ملف التحميل ناقص، ما بدى النظام بعد سنتين يتعلم:

[text]

"كلما اختفت شمعة صار بعدها تحرك قوي"
وهو بالحقيقة عم يتعلم أعطال الداتا.

وبالنسبة لـEpisode، إحنا عم نحمي فكرة مهمة جداً للمستقبل: FVG يظهر الآن وBOS بعد شوي وFlow بعده، لكن إذا كانوا عائلة سببية واحدة لازم نقدر نضيف الأعضاء بدون ما نعيد كتابة الحلقة القديمة.

هدول الاثنين بلوكرات حقيقيين. غيرهم ما رح أرجع أفتح H1 عليه. بعد RC1 إذا اتنفذوا بدون تناقض، القرار التالي BUILD S1.

MUF V1 S1 — FINAL DESIGN HARDENING H1

DESIGN PATCH — SIX BLOCKERS ONLY

Reference: MUF V1 S1 FINAL IMPLEMENTATION DESIGN

(deliverables/53_MUF_V1_S1_FINAL_IMPLEMENTATION_DESIGN.md, status

DESIGN ONLY — PENDING REVIEW).

Scope: exactly six blockers. No redesign outside them. No code, no tests, no MANIFEST, no S2.

This document supersedes the corresponding base sections where marked SUPERSEDES; everything else in the base design stands unchanged.

Status: DESIGN ONLY — READINESS REVIEW

CORRECTED SECTIONS

H1-1 — PRICE SOURCE SEMANTICS

(SUPERSEDES base §1.2 price labelling + §15 ACTUAL/PROXY columns)

Canonical price fields are NOT labelled ACTUAL/PROXY. Price is typed:

[text]

PUBLISHED_OHLC_FACT

= a published OHLC bar fact as delivered by an integrated source under its published source contract, carrying source_identity (source contract Schemalidentity), dataset_identity (observation domain, see H1-6), timeline_id/axis key binding, and the published bar record identity as provenance.

ACTUAL/PROXY remains reserved for contexts where an existing public source contract already defines those semantics — i.e. flow (ACTUAL = EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT). ACTUAL/PROXY may appear only on optional flow-context fields; never on price fields; the two are never merged (unchanged rule).

Source-general at the price-path contract level: the S1 price-path contract is parameterized over "integrated published-bar source". Binance Spot kline is the currently integrated source, not the ontology. No source semantics are invented; a future integrated source plugs into the same PUBLISHED_OHLC_FACT typing without changing S1 meaning.

No tick-path claim: published OHLC does NOT reconstruct the tick-by-tick market path. S1 claims only facts derivable from published bar facts (see H1-5: no claim that intermediate prices were traded).

H1-2 — LAWFUL OBSERVATION ADJACENCY

(SUPERSEDES base §8 adjacency wording; refines §2/§10)

Accepted observation stream (per timeline_id, axis): a sequence of published observations with lawful InformationKeys, ingested under a strict causal order contract:

```
each accepted key must be strictly greater than the previous accepted key
under S0 lawful comparison (same timeline, same axis);
out-of-order key (comparable but ≤ previous) → REJECT
SchemaViolation("S1_OUT_OF_ORDER_OBSERVATION") – malformed input is never
silently sorted;
duplicate key (equal key) → REJECT
SchemaViolation("S1_DUPLICATE_OBSERVATION_KEY") – a duplicate never creates
a primitive and never updates one (fail-closed; idempotent replay is not an
S1 contract);
incomparable keys → IncomparableInformationKeys (S0) – reject;
cross-timeline / cross-axis → InformationKeyViolation – reject;
same-batch ambiguity (same InformationBatchKey / same causal
coordinate): acceptance follows lawful key comparison only as stream
order; every chronology-dependent semantic between them stays
UNDEFINED(SAME_BATCH_ORDER_UNPROVEN) (base §10 stands).
Definition – OBSERVATION_ADJACENT(t-1, t) (EXACT, stream-causal):
```

t-1 and t are consecutive accepted observations of the same accepted stream under the lawful key order.

Adjacency is therefore created only by the lawful causal stream contract — never by row order, file order, or ingestion convenience. OBSERVATION_ADJACENT

zzz.txt — سجل المعادلات

asserts stream-causal consecutiveness only; it is explicitly not a market chronology claim beyond S0 key order and not a grid-continuity claim.

GRID_CONTIGUOUS remains a separate, independently proven property of the cadence/grid contract (expected keys per cadence; $\text{close_time} = \text{open_time} + \text{interval} - 1\mu\text{s}$ for the integrated source) — proven only when every expected grid key is observed (see H1-6). Never inferred from adjacency.

Permutation/out-of-order adversarial fixtures are mandatory (attacks A/B/D below).

H1-3 — RUNNING EXTREME TIE-SET (UNORDERED SEMANTIC SET)
(SUPERSEDES base §6 + §15.3 "key-sorted tie-set" wording)

Tie origins are an unordered semantic set: the semantic identity of `RUNNING_*_ORIGINS` is set equality over origin `InformationKeys`. Nothing about the tie-set semantics depends on order.

Any storage/serialization ordering is canonical serialization order only and is explicitly non-semantic — it is byte canonicalization for hashing (the same status `canonical_sha256` gives to sorted mapping keys), never market chronology. The base phrase "key-sorted tie-set" is retired because it can imply chronology.

Input permutation invariance: for the same legal facts, any permutation of representation/ingestion order of those facts yields the same semantic and hash identity of the tie-set (the canonical byte form is derived from the set, not from arrival order).

`deterministic_sequence` (and every key component) can never choose a "first/true" extreme. No downstream API may expose a winner: the published surface returns the immutable set only — no `.first`, no `.winner`, no tie-break, no "the origin". (`TIE_ORDER_CONTRACT = NOT_PROVEN.`)

H1-4 — DISPLACEMENT RUN: DEFERRED / OUT OF S1 V1
(SUPERSEDES base §5 entirely; removes run entries from §15/§18/§19 surface)

Default decision applied: DEFERRED / OUT OF S1 V1.

`DisplacementRunRecord` / `DisplacementRunExtentRecord` and the whole segment-identity section are REMOVED from S1 V1 schemas, build plan, and certification surface.

Reason: (a) no required dependency in accepted S1 scope cannot be represented by adjacent primitives/descriptors — P1-P6 and the factual descriptors are complete without it; (b) S1 must not introduce unnecessary segmentation ontology before S2+; (c) a "maximal" forming run has future-sensitive extent semantics (its extent is only known later), conflicting with S1's adjacent-fact-closed discipline.

Not replaced by any other run/segment abstraction. S1 V1 has zero segmentation entities. Entity/segment identity contracts arrive with S2+ under separate design and authorization.

What remains identity-bearing in S1 V1: the primitive/record identities (§15 schemas, S0 hashing) only.

H1-5 — INTRABAR PATH LOWER BOUND — DISCRETE PROOF (RETAINED)
(SUPERSEDES base §4 proof statement and wording)

The lower bound is retained, restated without any continuity assumption:

Class of sequences. Let $\mathcal{O}(O, H, L, C)$ be the set of all finite ordered observed/executed price sequences ($x_0 = O, x_1, \dots, x_n = C$), $n \geq 1$, that are consistent with the published OHLC facts: first value O , last value C , attained maximum exactly H , attained minimum exactly L . Additional

intermediate observations are allowed (arbitrary refinement).

Claim (BOUND). For every sequence in $\mathcal{O}(H,L,C)$:

[text]

$$TV(x) = \sum |x_i - x_{i-1}| \geq (H - L) + \min(|O-H| + |L-C|, |O-L| + |H-C|)$$

Proof. Any such sequence contains indices i_H (value H) and i_L (value L) — both extrema must be visited. If $i_H < i_L$, split the sum at those two visits: $TV \geq |O-H| + |H-L| + |L-C|$ (triangle inequality on each sub-segment; the middle sub-segment has total variation $\geq |H-L|$). If $i_L < i_H$, symmetrically $TV \geq |O-L| + |L-H| + |H-C|$. Edge cases where O or C coincide with an extremum collapse the corresponding term to zero. In both orders TV is at least the shorter of the two visitation routes, i.e. the stated LB. Tightness: the three-leg route sequence (monotone legs through the two extrema in the shorter order) is itself an admissible discrete sequence and attains LB; hence LB is the greatest valid lower bound over $\mathcal{O}$.

Explicit non-claims. No continuity of price is assumed. No claim is made that any intermediate price was traded — the bound holds for every refinement because total variation only grows under refinement. The published OHLC does not reveal which sequence in $\mathcal{O}$ occurred.

Consequences.

INTRABAR_PATH_LENGTH_EXACT remains UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY).

INTRABAR_PATH_LENGTH_LOWER_BOUND = BOUND, carried with `bound_basis = "DISCRETE_TV_TRIANGLE_INEQUALITY"` in the record/DescriptorSpec.

INTRABAR_PATH_LENGTH_UPPER_BOUND remains UNAVAILABLE (unbounded refinement/oscillation).

If a future source provides trade-level observed sequences, exact path becomes a separate source-typed metric — not S1 V1.

H1-6 — GRID OBSERVATION GAP $\neq$ MISSING MARKET BAR

(SUPERSEDES base §8.1/§8.3 naming + §15.5 + related error names)

Rename/redefine. MissingIntervalRecord is renamed

ExpectedGridKeyNotObservedRecord. Its claim type is exactly:

[text]

EXPECTED_GRID_KEY_NOT_OBSERVED (per expected grid key)

GRID_OBSERVATION_MISSING (typed unavailability reason)

and nothing stronger.

Claim scope. The record asserts only: an expected grid key was not observed within a specific observation domain

(`source_identity` $\times$ `dataset_identity` $\times$ `timeline_id` $\times$ `cadence_contract`). It must NOT claim — and its schema/text must never state — that "the market/exchange did not produce a bar". Missingness is data/source observation missingness, and it is never directional market evidence.

Required provenance fields (identity-bearing): `source_identity`, `dataset_identity`, `timeline_id`, `cadence_contract_identity` (interval + close-time rule), `expected_grid_keys` (the span); plus non-identity `detection_key` (the lawful InformationKey at which non-observation within the dataset domain is proven).

Adjacency effects. `GRID_CONTIGUOUS` = false/unavailable whenever an expected key in the span is not observed (asserting contiguity raises SchemaViolation("S1_GRID_CONTIGUITY_UNPROVEN") with the typed reason `GRID_OBSERVATION_MISSING`). `OBSERVATION_ADJACENT` remains factual

zzz.txt — سجل المعادلات

between consecutive lawful observations across the gap (H1-2) and is labelled as such.

Unchanged fail-closed rules: no synthetic bar; no forward-fill; no interpolation; gap spans propagate only as typed
NOT_APPLICABLE / GRID_OBSERVATION_MISSING states.

IMPACTED SCHEMA DELTAS

Schema / element Delta

price fields everywhere (15.1-15.4) typed PUBLISHED_OHLC_FACT + source_identity/dataset_identity; ACTUAL/PROXY removed from price; ACTUAL/PROXY only on optional flow-context fields (never required, never merged)

ObservedDisplacementRecord (15.1) adjacency_kind semantics now cite the H1-2 stream-causal definition; gap_ref → ExpectedGridKeyNotObservedRecord ref or NOT_APPLICABLE; price typing per above; identity fields unchanged

BarFactDerivativesRecord (15.2) intrabar_path_length_lower_bound gains bound_basis = "DISCRETE_TV_TRIANGLE_INEQUALITY" (H1-5); exact path stays

UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY); upper bound stays UNAVAILABLE; price typing per above

RunningExtremeSnapshotRecord (15.3) running_high_origins / running_low_origins = unordered semantic set; canonical serialization order documented as non-semantic; no winner/first API exists; hash identity permutation-invariant (H1-3)

DescriptorValueRecord (15.4) input typing per above; no other field change

MissingIntervalRecord (15.5) renamed ExpectedGridKeyNotObservedRecord; + identity provenance source_identity, dataset_identity, cadence_contract_identity, expected_grid_keys; reason ∈ {EXPECTED_GRID_KEY_NOT_OBSERVED}; typed gap unavailability = GRID_OBSERVATION_MISSING (H1-6)

DisplacementRunRecord / DisplacementRunExtentRecord (base §5) REMOVED — DEFERRED (H1-4); episode/transition/explanation schemas keep membership/refs to primitive/record facts only

CausalEpisodeRecord / MarketStateTransitionRecord / ExplanationRecord (15.6/§12) unchanged fields and states; only reference set restricted to the surviving S1 V1 records

error/reason vocabulary MISSING_GRID_BAR retired → EXPECTED_GRID_KEY_NOT_OBSERVED (record reason) / GRID_OBSERVATION_MISSING (typed unavailability); + S1_OUT_OF_ORDER_OBSERVATION, S1_DUPLICATE_OBSERVATION_KEY, S1_GRID_CONTIGUITY_UNPROVEN;

UNSUPPORTED_SOURCE_SEMANTICS now also fires on ACTUAL/PROXY applied to price fields

UPDATED ADVERSARIAL ATTACKS

Base matrix (§17) stands with these amendments: row 2 uses the discrete jump sequence fixture (attack F); row 4 wording uses the unordered-set identity (attack D); row 5 wording is dataset-domain (attack C); row 16 is scoped to flow-only ACTUAL/PROXY (attack E). New mandatory attacks:

Attack Violated contract Minimal fixture Expected rejection / state Design gate
A shuffled input rows change lawful adjacency silently H1-2 legal stream keys $k_1 < k_2 < k_3$ ingested as k_1, k_3, k_2 SchemaViolation("S1_OUT_OF_ORDER_OBSERVATION") at $k_3 \rightarrow k_2$ – adjacency set unchanged; no silent sort strict-monotonic ingestion contract + test
B duplicate InformationKey creates a primitive H1-2 same key ingested twice
SchemaViolation("S1_DUPLICATE_OBSERVATION_KEY"); no second primitive, no update duplicate rejection test
C missing expected grid key reported as "market bar absent" H1-6 dataset domain missing grid key 6 record says EXPECTED_GRID_KEY_NOT_OBSERVED within the declared domain only; any "market bar absent" wording/field → SchemaViolation schema claim-scope test + wording lock
D same tie-set under different ingestion ordering has different identity H1-3 tie-set { k_5, k_9 } built/serialized in both orders same semantic identity and same canonical hash; no winner exposed permutation-invariance test + no-winner API check
E price field uses flow-style ACTUAL/PROXY H1-1 price field labelled ACTUAL or a merged provenance SchemaViolation("UNSUPPORTED_SOURCE_SEMANTICS") PUBLISHED_OHLC_FACT typing test (price fields carry no ACTUAL/PROXY)
F intrabar lower bound tested via continuity H1-5 fixture computes LB on a discrete jump sequence (0,H,L,C as attained values) and on a refined sequence with extra points both give $TV \geq LB$; no continuity premise anywhere in the metric contract; any continuous-path claim → SchemaViolation bound_basis field + discrete-proof unit fixture
G DisplacementRun/segment sneaks back into S1 V1 H1-4 search/AST over proposed S1 modules for

zzz.txt — سجل المعادلات

DisplacementRun, segment/run entity classes, or run-organizing logic zero hits – nothing planned; any hit is a design violation build-plan lock + AST/search gate in the future build
 UPDATED BUILD PLAN (deltas only)
 File Delta
 src/trading_system/market_understanding/price_path.py + strict-monotonic ingestion contract (reject duplicates/out-of-order/incomparable/cross-timeline – never sort); + RunningExtremeOrigins as unordered semantic set with non-semantic canonical serialization; + bound_basis = "DISCRETE_TV_TRIANGLE_INEQUALITY" on the LB metric; explicitly contains NO run/segment implementation (attack G)
 src/trading_system/market_understanding/path_schemas.py ExpectedGridKeyNotObservedRecord replaces MissingIntervalRecord (+ domain provenance fields); price typing PUBLISHED_OHLC_FACT; ACTUAL/PROXY vocabulary restricted to flow-context fields; run/segment schemas deleted (H1-4)
 tests/test_muf_sl_price_path.py + permutation/out-of-order fixtures (A), duplicate-key (B), tie-set permutation identity (D), discrete LB fixture (F)
 tests/test_muf_sl_path_schemas.py + dataset-domain gap wording (C), price-typing scan (E), AST/search no-segment gate (G)
 Complexity and public-API reuse are unchanged from base \$18 (0(1) amortized per bar; S0/CLOSED APIs only). S0 remains untouched.

REMAINING OPEN / NOT_CONFIGURED (explicit)

DEFERRED (design decision recorded):

DisplacementRun / any run or segment ontology → out of S1 V1, S2+ under separate design and authorization (H1-4). No replacement abstraction exists in S1 V1.

NOT_CONFIGURED at S1 V1 (unchanged from base, restated):

wave semantics; turning-point detection/quality/importance; hierarchy; $\alpha/\beta/\gamma/\delta$ representations; policies/qualifications; regime thresholds; narrative population beyond the four explanation states; statistical independence (RESEARCH-DEBT-024 OPEN); merging of ACTUAL/PROXY; any volume/flow requirement; final-normalized/future-normalized descriptors; tick-by-tick path reconstruction claims; trade-level exact intrabar path (future source-typed metric under separate authorization).

OPEN:

base design remains DESIGN ONLY — PENDING REVIEW;

this H1 hardening patch is DESIGN ONLY — READINESS REVIEW.

STATUS

DESIGN ONLY — READINESS REVIEW

Six blockers corrected; schemas, errors, attacks, and build plan cross-checked and updated without scope expansion. No code. STOP.

gpt-5.5-instant

الحالة الرسمية: نحنا عند S1 DESIGN ONLY — PENDING REVIEW. ما في BUILD بعد. راجعت التصميم ككل، وبدل ما نفوت بقطارة تعديلات، عملت عليه مراجعة readiness كاملة من الآن. حكمي: التصميم قوي كأساس، لكنه غير جاهز للبناء بعد بسبب 6 BLOCKERS مفاهيمية متشابكة. هدول لازم ينحلوا بجولة تصميم واحدة، وبعدها إما BUILD أو رفض واضح. ما عندي مصلحة أرجعه عشان أسماء أو تجميل.

في شغلات انحسمت صح وممتازة هندسياً: فصل close-space path عن intrabar path صحيح؛ عدم اختراع H-before-L؛ lower bound المقترح للمسار داخل الشمعة صحيح رياضياً تحت فرض continuous path الذي يزور H ول؛ running extreme بقي fact وليس turning point؛ gaps انقسمت observation-adjacent عن grid-contiguous؛ وما في threshold/quantile/window.

لكن المشاكل الستة التالية لو تركناها، رح نبني فوق تعريفات ممكن تكون خاطئة من الأساس.

أخطر مشكلة: S1 خلط "مصدر Binance" مع "حقيقة Price Path عامة".
 التصميم يقول إن المدخل:

[text]

open/high/low/close

= Binance Spot kline published bar fact

ثم يسمى السعر:

[text]

ACTUAL published bar fact

هون في خلط ontology.

الـ OHLC المنشور Canonical factual بالنسبة لمصدر الشمعة، نعم. لكن ACTUAL/PROXY في دستور المشروع أصلاً فصلناه لمعنى flow/source semantics، خصوصاً executed initiated flow مقابل OHLCV proxy.

السعر نفسه لا يحتاج أن نسميه ACTUAL مقابل PROXY بهذا المعنى.

وS1 المفروض تكون MUF price-path عامة، مش Binance-only للأبد.

الأصح:

[text]

PriceFact

source_semantics =

PUBLISHED_OHLC_FACT

وبروفينانس المصدر يحدد Binance الآن.

أما:

[text]

ACTUAL / PROXY

يبقى للـ flow/context إذا دخل لاحقاً.

وإلا بعدين يصير عندنا:

[text]

price = ACTUAL

flow = ACTUAL

بس الكلمتين لهما معنى مختلف، وهذا باب semantic laundering.

هذا BLOCKER لأنه يدخل في §15 schemas كلها.

تعريف adjacency فيه خلط بين "صفين متتاليين في بياناتنا" و"شمعتين متتاليتين في grid".
التصميم يقول:

[text]

$C_t - C_{t-1}$

وبعدين عند gap يسميها:

[text]

OBSERVATION_ADJACENT

هذا جيد، بس مين يثبت observation adjacency؟

لو وصل input:

[text]

bar positions:

1, 5, 3

هل نرتبهم أولاً؟

هل ingestion order هو observation order؟

هل InformationKey order؟

هل input لازم أصلاً يكون monotonically ordered؟

التصميم ما حسمها.

وهاي مهمة لأننا بنينا المشروع كله على:

[text]

row order $\neq$ market chronology

ما بصير S1 يعمل:

[Python]

previous_row
current_row
ويعتبرها factual adjacency بدون عقد.

لازم يكون عندنا تعريف صريح:

[text]

OBSERVATION_ADJACENT

=

consecutive elements under a lawful causal ordering contract
within one timeline/axis
أو نفرض input stream نفسه لديه monotonic InformationKey contract ونرفض out-of-order.
غير هيك، row shuffle، غير primitives.

BLOCKER.

مشكلة "key-sorted set": Running Extreme tie-set: قد تعيد إدخال chronology من sequence.
التصميم يقول:

[text]

immutable, key-sorted set of InformationKeys
with no semantic order
إذا هي set دلاليًا، ممتاز.

لكن "key-sorted" خطيرة لأن InformationKey فيه deterministic_sequence.
نحن حرفياً أغلقنا في S0 أن:

[text]

deterministic_sequence $\neq$ market chronology
إذا الترتيب فقط canonical serialization، لازم نسميه:

[text]

canonical storage order only
ولا يستطيع أي downstream consumer استخدام index 0 على أنه first high.
والأقوى: identity/hash لل tie-set لازم يكون order-insensitive دلاليًا.
مثلاً:

[text]

{K1,K2}

نفس الحقيقة مهما كان ترتيب الإدخال.

بدا contract صريح لذلك.

DisplacementRunRecord لا أنصح بإدخاله في S1 أصلاً.
التصميم يقول:

[text]

maximal run of same-sign close displacements
كلمة maximal هون مخادعة سببياً.

تخيل:

[text]

102 $\rightarrow$ 101 $\rightarrow$ 100

عند 102 عندك run UP.

هل هو maximal؟

ما بتعرف، لأن المستقبل قد يكون:

[text]

103 $\rightarrow$

فصار أطول.

البيلدر حاول يحلها بـ identity origin-only و extent later، وهذا سببي، لكن الكيان نفسه يبدأ يقرنا من segmentation قبل ما نحتاجها.

والأهم:

[text]

same-sign run

هو أول interpretation لحركة السعر، ولو factual.

هل نحتاجه فعلاً في S1؟ لا.

S1 يمكن أن يعيش بالكامل على:

[text]

adjacent displacement

bar derivatives

running aggregates

وتترك S2/S3 runs إذا أثبتنا الحاجة.

هذا ليس مجرد preference عندي بسبب كلمة maximal وكون endpoint لا يعرف إلا لاحقاً. ممكن نصممه صحيح، لكن ما في أي dependency حالية تحتاجه. إدخاله الآن يزيد ontology بلا ضرورة.

حكمي: احذفه من S1 V1. إذا البيلدر ثبت dependency لازمة، يذكرها.

الـ intrabar lower bound صحيح، لكن فرض "continuous intrabar path" نفسه غير مثبت للمصدر بالشكل المكتوب. المعادلة:

[text]

LB =

(H-L)

+

min(

$|O-H| + |L-C|$,
 $|O-L| + |H-C|$

(

رياضياً صحيحة لمسار continuous يبدأ O وينتهي C ويزور H وL.

لكن السوق discrete trades.

والـ kline OHLC هو aggregate لصفقات discrete.

ممكن السعر يقفز من trade إلى trade:

[text]

105 → 100

بدون أن يتداول 101,102,103,104.

يعني كلمة continuous ليست حقيقة مصدرية.

هل lower bound على total variation لسلسلة discrete التي تضم O,H,L,C أيضاً صحيح؟ نعم، غالباً نقدر نثبت نتيجة مشابهة بدون فرض continuity: أي sequence يبدأ O وينتهي C ويحتوي H و total variation $\leq L$ أقصر route visiting both extrema.

وهذا أحسن.

إذن لازم يشيل فرض continuity ويبرهنها على:

[text]

any ordered price sequence consistent with the OHLC extrema

من دون افتراض أن الأسعار الوسيطة كلها حدثت.

هذا تصحيح مهم لأننا ممنوع نضيف market microstructure assumption بلا مصدر.

أكبر ثغرة تشغيلية: MissingIntervalRecord يدعي غياب bar، لكن "غياب صف" لا يثبت "غياب bar من السوق". التصميم يقول:

[text]

reason = MISSING_GRID_BAR
إذا dataset ناقصة، الذي نعرفه:

[text]

EXPECTED GRID OBSERVATION ABSENT FROM THIS INPUT
لكن هل البورصة فعلاً لم تنتج bar؟
مختلف تماماً.

Binance 24/7 قد يكون لدينا cadence contract، لكن غياب الملف يمكن يكون download defect.
إذن الاسم لازم ما يدعي غياب حدث سوقي.

الأقوى:

[text]

GRID_OBSERVATION_MISSING
أو:

[text]

EXPECTED_GRID_KEY_NOT_OBSERVED
وبروفينانس:

[text]

within this source/dataset/timeline
ولا نقول market bar did not exist.
وهذا مرتبط بفلسفة المشروع كلها:

[text]

missing data $\neq$ market fact
BLOCKER لأنه لو اتسمى downstream MISSING_GRID_BAR، قد يتعامل معه كحقيقة أن السوق لم يكن عنده شمعة.

الخلاصة:

[text]

```
S1 Design
├─ OHLC ambiguity ✓
├─ Close-space path ✓
├─ Path lower-bound mathematics ● continuous assumption إزالة
├─ Running extremes ● tie-set semantics
├─ Gaps ● missing observation  $\neq$  missing market bar
├─ Adjacency ● needs lawful ordering contract
├─ Price ACTUAL/PROXY ontology ● needs correction
├─ DisplacementRun ● remove/defer in V1
├─ Performance ✓ conceptually
├─ Future append ✓ conceptually
└─ BUILD. ما رح نعطي
```

وينفس الوقت ما رح نرجع بخمس جولات. هاد Design Patch واحد جامع، وبعده أنا أحكم BUILD readiness مباشرة. ابعت للبيلد:

[text]

MUF V1 S1 — FINAL DESIGN HARDENING H1
SIX BLOCKERS ONLY — DESIGN PATCH — NO CODE

Reference:

MUF V1 S1 FINAL IMPLEMENTATION DESIGN

Status:

DESIGN ONLY — PENDING REVIEW

This patch addresses exactly six blockers.

No redesign outside them.

No code.

No tests.

No MANIFEST.

No S2.

1 — PRICE SOURCE SEMANTICS

Remove ACTUAL/PROXY labels from canonical price fields unless an existing public source contract explicitly uses those semantics for price.

Define price source semantics separately, e.g. a typed:

PUBLISHED_OHLC_FACT

with source/provenance identity.

ACTUAL/PROXY remains reserved for contexts such as flow where those semantics already have a defined contract.

S1 remains source-general at the price-path contract level:

Binance is the currently integrated source, not the ontology.

No claim that published OHLC reconstructs tick-by-tick market path.

2 — LAWFUL OBSERVATION ADJACENCY

Define OBSERVATION_ADJACENT exactly.

Row order/file order must not create adjacency.

Choose a causal contract such as:

- input stream must be strictly monotonic under lawful comparable

InformationKeys within one timeline/axis,
and out-of-order input is rejected,

OR an equally strict existing CLOSED stream-order contract.

Do not silently sort malformed input.

Define behavior for:

- duplicate key,
- out-of-order key,
- incomparable key,
- cross-timeline,
- same-batch ambiguity.

GRID_CONTIGUOUS remains separately proven through cadence/grid contract.

Add permutation/out-of-order adversarial fixtures.

3 — RUNNING EXTREME TIE-SET

Define tie origins as an unordered semantic set.

Any storage ordering is canonical serialization order only,
never market chronology.

Requirements:

- input permutation does not change semantic/hash identity of the tie-set

where the same legal facts are represented.

- deterministic_sequence cannot choose a "first true extreme".

- no downstream API may expose a winner from a tie-set.
- canonical storage order must be explicitly non-semantic.

Do not call it key-sorted in a way that implies chronology.

4 — DISPLACEMENT RUN

Remove DisplacementRunRecord from S1 V1 unless you can prove a required dependency in accepted S1 scope that cannot be represented by adjacent primitives/descriptors.

Default decision:
DEFERRED / OUT OF S1 V1.

Reason:
S1 should not introduce unnecessary segmentation ontology before S2+, and a "maximal" forming run has future-sensitive extent semantics.

If deferred:
remove it from schemas/build plan/certification surface.

Do not replace it with another run/segment abstraction.

5 — INTRABAR PATH LOWER BOUND PROOF

Retain the lower bound only if it can be proved WITHOUT assuming a continuous price path.

State it over:

any ordered observed/executed price sequence consistent with the published OHLC facts, starting at O, ending at C, and containing extrema H and L.

Prove the total variation lower bound through triangle inequality / required visitation of both extrema.

Do NOT claim intermediate prices were traded.

If that proof cannot be stated cleanly:
remove the lower-bound metric from S1 V1 and leave exact path UNAVAILABLE.

No microstructure assumption may be invented.

6 — MISSING OBSERVATION $\neq$ MISSING MARKET BAR

Rename/redefine MissingInterval semantics.

S1 may prove only something like:

EXPECTED_GRID_KEY_NOT_OBSERVED

or

GRID_OBSERVATION_MISSING

within a specific:

source

dataset

timeline

cadence contract.

It must NOT claim:

the market/exchange did not produce a bar.

Required provenance fields must identify the dataset/source observation domain.

GRID_CONTIGUOUS:

false/unavailable when an expected key is not observed.

OBSERVATION_ADJACENT:

may still be factual between consecutive lawful observations.

No forward-fill/synthetic bar.

Missingness is data/source missingness, not directional market evidence.

CROSS-CHECKS

After correcting the six items, update all affected:

- schemas
- error names/reasons
- adversarial matrix
- build plan
- certification wording

without expanding scope.

Explicitly add attacks:

- A) shuffled input rows cannot change lawful adjacency silently.
- B) duplicate InformationKey does not create a primitive.
- C) missing expected grid key is not reported as "market bar absent".
- D) same tie-set under different ingestion ordering has same semantic identity.
- E) no price field uses flow-style ACTUAL/PROXY semantics.
- F) intrabar lower bound test uses a discrete jump sequence, not continuity.
- G) search/AST confirms no DisplacementRun/segment implementation is planned for S1 V1.

DELIVERY

Return:

MUF-V1-S1-FINAL-IMPLEMENTATION-DESIGN-H1

Status:

DESIGN ONLY — READINESS REVIEW

Include only:

- corrected six sections,
- impacted schema deltas,
- updated attacks,
- updated build plan,
- explicit statement of remaining OPEN/NOT_CONFIGURED items.

No code.

.STOP

وحتى تكون الصورة عندك واضحة: نحن ما عم نعيد اختراع S1. تقريباً بنيتة صارت موجودة. هداول الستة عبارة عن تنظيف حدود الحقيقة قبل الكود.

أخطر اثنين عندي هما:

[text]

row adjacency $\neq$ market chronology

و:

[text]

missing observation $\neq$ missing market event

لو بنينا وهما غلط، لاحقاً النظام ممكن يعتبر عطل تحميل بيانات «gap بالسوق»، أو يعتبر ترتيب CSV chronology حقيقية. هداول ما بسايرك فيهم حتى لو بدنا نسرع.

أما بعد H1، إذا انغلقوا وما ظهر تناقض جديد من نفس التصحيحات، أعطي BUILD. ما رح أفتح جولة ثانية لأفكار تجميلية.

MUF V1 — S1 FINAL IMPLEMENTATION DESIGN

PRICE PATH PRIMITIVES — DESIGN ONLY

Task: DESIGN S1 ONLY (OWNER AUTHORIZATION). S0 is CLOSED.

Official baseline: MANIFEST

899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84

(196/196 OK) · project 1139/1139 · field_runner 36/36 · MUF S0 CLOSED.

No S2 or later stage is authorized.

Status: DESIGN ONLY — PENDING REVIEW

Deliverable: this document only. No code, no tests, no MANIFEST, no docs closure.

0 — CORE PRINCIPLE (binding on every section)

text

PRICE PATH FACT $\neq$ TURNING POINT $\neq$ WAVE $\neq$ MARKET IMPORTANCE

S1 describes what the observed market path did — factually, causally, at declared availability. S1 must NOT decide which turning point is important, which wave is "real", which hierarchy is correct, which representation $\alpha/\beta/\gamma/\delta$ wins, or any predictive significance.

Standing semantics preserved verbatim: TIE_ORDER_CONTRACT = NOT_PROVEN; ambiguous O/C stays ambiguous (convention witnesses are never canonical OHLC); ZERO LOOKAHEAD; Origin $\neq$ Availability; kline = published bar fact; FAIL CLOSED; PROXY $\neq$ ACTUAL and never merged.

1 — INPUT AUTHORITY

S1 consumes only existing PUBLIC CLOSED market/timeline contracts. No source semantics are invented.

1.1 Required factual inputs (exact)

Input Contract source Semantics

open, high, low, close published bar fact (CLOSED Binance Spot kline OHLC source contract) strict finite decimals; canonical O/C from the kline source exclusively; open_ambiguous / close_ambiguous stay ambiguous; close_time = open_time + interval – 1 μ s cadence as published

InformationKey / timeline identity research.information_time.InformationKey (information_key_version, timeline_id, bar_position, event_time_utc, information_phase, deterministic_sequence); comparison coordinate (bar_position, phase_rank, sequence); timeline identity is part of every identity axis semantics InformationAxis POSITIONAL (bar_position only, event_time_utc is None) or TIME_INDEXED (timezone-aware UTC required)

source provenance S0 Schemalidentity + compute_record_identity every consumed bar fact referenced by its published record identity + schema identity; no parallel hashing

1.2 Volume / flow

NOT required to describe price path. If referenced at all it is an optional contextual reference in a separate, clearly-typed field set:

flow_kind $\in$ {ACTUAL, PROXY} where ACTUAL means only

EXECUTED_INITIATED_FLOW_FROM_BINANCE_SPOT per the source's own semantics;

ACTUAL and PROXY are stored in disjoint fields, never summed, never merged;

zzz.txt — سجل المحادثات

no price-path primitive, descriptor, or schema record may depend on volume/flow. Missing volume is TypedState.NOT_CONFIGURED, never a failure of the price-path layer.

2 — DEFINITION: PRICE PATH PRIMITIVE

A PricePathPrimitive is a factual function of a closed set of already published bar facts whose availability keys are all $\leq$ the primitive's declared availability key, with:

- no swing threshold, quantile, ATR multiple, or fixed bar window;
- no future confirmation of any kind;
- no visual/semantic importance attribution;
- deterministic value (or deterministic typed UNAVAILABLE);
- metric_semantics $\in$ {EXACT, BOUND, PROXY, UNAVAILABLE} on every metric.

2.1 The precise S1 primitive schema (final)

Atom: completed bar fact $B_t = (O_t, H_t, L_t, C_t)$ at its published availability key. Derived primitives — all adjacent-fact-closed:

Primitive Formula Semantics Availability Why knowable there

P1 CLOSE_DISPLACEMENT(t) $C_t - C_{\{t-1\}}$ (signed) EXACT, close-space close key of t (the later key) only closes of two observed bars, both published

P2 CLOSE_PATH_STEP(t) $|C_t - C_{\{t-1\}}|$ EXACT, close-space close key of t same

P3 BAR_RANGE(t) $H_t - L_t (\geq 0)$ EXACT (span fact — not a path length) close key of t completed bar extremes

P4 OPEN_CLOSE_DISPLACEMENT(t) $C_t - O_t$ (signed; body) EXACT — named separately from P1 close key of t completed bar

P5 UPPER_WICK(t) / LOWER_WICK(t) $H_t - \max(O_t, C_t) / \min(O_t, C_t) - L_t$ EXACT close key of t completed bar

P6 RUNNING_HIGH_SO_FAR(T) / RUNNING_LOW_SO_FAR(T) + origin tie-set max/min over completed bars $\leq T$ EXACT value; tie-set factual (§6) T's declared snapshot key prefix of published bars only

Each primitive's identity is causal (§5). Any metric that would require intrabar chronology is not a primitive here — see §3/§4.

3 — OHLC INTRABAR AMBIGUITY (mandatory)

OHLC does NOT reveal whether $O \rightarrow H \rightarrow L \rightarrow C$ or $O \rightarrow L \rightarrow H \rightarrow C$ (or any other order

visiting the extremes) occurred.

KNOWN: H occurred, L occurred, O occurred, C occurred (within the bar's interval).

UNKNOWN: the relative chronology of H vs L (and of the extremes vs O/C interior) unless the source proves it — the kline source does not.

Contract:

S1 never constructs a fictitious intrabar traversal. No OHLC order is chosen for convenience.

Any path quantity that depends on high-before-low order is

UNDEFINED(AMBIGUOUS_INTRABAR_CHRONOLOGY) (typed state), or

UNAVAILABLE(metric_semantics=UNAVAILABLE, reason=AMBIGUOUS_INTRABAR_CHRONOLOGY).

Lawful bounds are permitted only where mathematically defensible for all consistent orders (§4).

Requesting the chronology itself is a deterministic error (§16).

4 — PATH LENGTH (proof-based semantics)

Question: what does "distance travelled" mean when only OHLC is known?

Forbidden: $|O-H| + |H-L| + |L-C|$ (assumes chronology) — and every other single traversal formula.

Metric Value Semantics Proof status

close_to_close_path_contribution(t) $|C_t - C_{t-1}|$ EXACT — explicitly close-space path step, never called "the" path definitional

observed_close_path_length(T) $\sum_{i \leq T} |C_i - C_{i-1}|$ (close series) EXACT as close-space metric; cumulative sum telescopes incrementally definitional; label forbids intrabar interpretation

bar_range(t) $H_t - L_t$ EXACT span fact (not a path) definitional

intrabar_path_length_exact(t) — UNAVAILABLE (AMBIGUOUS_INTRABAR_CHRONOLOGY) chronology unknown (§3)

intrabar_path_length_lower_bound(t) $LB = (H-L) + \min(|0-H| + |L-C|, |0-L| + |H-C|)$ BOUND (exact lower bound) Proof: any continuous intrabar path from 0 to C must visit both H and L; by the triangle inequality its total variation is at least the shorter of the two routes $0 \rightarrow H \rightarrow C$, $0 \rightarrow L \rightarrow C$; both routes are achievable by monotone legs, so LB is attained. Holds for every consistent order.

intrabar_path_length_upper_bound(t) — UNAVAILABLE (unbounded: arbitrary oscillation inside $[L, H]$ is consistent with the same OHLC) no finite bound is defensible

Every path metric is declared EXACT | BOUND | PROXY | UNAVAILABLE in its DescriptorSpec/record. Bounds are published as bounds — never promoted to exact values.

5 — SEGMENT IDENTITY

S1 primitives are naturally closed by currently observed adjacent facts

(§2). Optional secondary entity: the DisplacementRunRecord (a maximal run of same-sign close displacements) — explicitly NOT a wave, NOT a turning point, NOT a swing (the CLOSED 2.1A chain is out of S1 scope entirely).

Identity contract (causal and immutable):

Element Rule

origin the first adjacent pair of the run (both bar keys known at the run's birth)

identity-defining fields schema_identity, origin_pair_keys (both InformationKeys incl.

timeline/axis/sequence), direction_sign, run_schema_version — nothing else. The final endpoint is never identity and never enters identity retroactively.

extent (non-identity) published later as DisplacementRunExtentRecord (a new observation/event when the run closes) — SUPERSEDE/extent never modifies the run's identity or earlier facts

availability InformationKey existence: availability of the second close of the origin pair; extent: availability of the closing bar's key

provenance CausalRecordReference list to contributing bar-fact record identities

(validate_reference_at/type at every use)

schema/version/domain hash compute_schema_identity(schema_family="MUF_S1_PRICE_PATH", schema_version="V1", domain=...) + compute_record_identity — reuse S0 identities; no parallel hashing

6 — RUNNING EXTREMES (RUNNING_EXTREME, never TURNING_POINT)

text

RUNNING_EXTREME $\neq$ TURNING_POINT. A running extreme is a prefix fact.

RUNNING_HIGH_SO_FAR(T) = $\max(H_i \mid i \leq T, i \text{ completed})$; low analogously.

Never described as peak/trough/confirmed extreme/turning point. Public schema names use RUNNING_* exclusively (naming is part of the contract).

Ties: when a new bar ties the running value, the tie-set is retained:

RUNNING_HIGH_ORIGINS(T) = the set of bar keys attaining the running max, represented as an immutable, key-sorted set of InformationKeys with no semantic order ("first is the true high" is forbidden — row order is not market chronology; TIE_ORDER_CONTRACT = NOT_PROVEN). All tied origins are kept (no ambiguity-set collapsing to a single "winner").

In TIME_INDEXED space, lawful timestamps order observations, but a price tie remains a tie: both origins stay in the set.

Forming-bar partials (BAR_PRE_CLOSE phase) are a separate typed fact (FORMING_BAR_RUNNING_HIGH etc.), never conflated with completed-bar

running extremes (availability per phase; attack 8/attack 12 gate this).

7 — DIRECTION (observed displacement only)

Fact Definition Allowed names

CLOSE_TO_CLOSE_DISPLACEMENT(t) $C_t - C_{t-1}$ (signed, EXACT) as named

CLOSE_TO_CLOSE_DIRECTION(t) UP iff displacement > 0; DOWN iff < 0; FLAT iff == 0 UP/DOWN/FLAT only

OPEN_CLOSE_DISPLACEMENT(t) $C_t - O_t$ (signed, EXACT) — separately named, body direction separate from close-to-close direction as named

Forbidden vocabulary anywhere in S1 public API/records: trend, bullish,

bearish, structure, impulse, correction. Direction describes an

observed displacement and nothing else. Equal closes → FLAT (factual zero),

no interpretation.

8 — GAPS / MISSING BARS (mandatory fail-closed)

If bar at grid position t is missing and t+1 is observed:

No synthetic bar. No forward fill. No interpolation. The missing interval stays explicit as MissingIntervalRecord (expected grid positions, cadence identity, reason MISSING_GRID_BAR).

ADJACENCY_KIND ∈ {GRID_CONTIGUOUS, OBSERVATION_ADJACENT} is typed on every adjacent-pair fact.

The numerical $C_{t+1} - C_t$ is computable but is published only as

OBSERVATION_ADJACENT displacement; a GRID_CONTIGUOUS claim across the gap is UNDEFINED(MISSING_GRID_BAR) and asserting it is a deterministic

error (§16).

Grid membership uses the existing cadence/grid contracts (published kline cadence; close_time = open_time + interval - 1μs; InformationKey bar_position in POSITIONAL space). No new grid semantics are invented.

Cumulative close-space path length across the gap: the OBSERVATION_ADJACENT step is included only in the observation-adjacent accumulator; the grid-contiguous accumulator records UNDEFINED(MISSING_GRID_BAR) for the gap span. Both accumulators are distinct DescriptorSpecs.

9 — TIME-INDEXED vs POSITIONAL

POSITIONAL TIME_INDEXED

timestamps never invented; event_time_utc is None timezone-aware UTC required (S0 InformationKey contract already enforces)

bar-count duration EXACT (factual) EXACT (factual)

wall-clock duration UNAVAILABLE(NOT_TIME_INDEXED) EXACT from lawful timestamps

axis conversion never silent; cross-axis duration/metric comparison → IncomparableInformationKeys / reject idem

Axis is part of identity; every record declares its axis-bound keys. Naive or invented timestamps are rejected at key construction (S0 behavior), not repaired.

10 — SAME INFORMATION BATCH

Using S0 InformationBatchKey semantics: facts co-visible in one batch have no proven A-before-B order. Mechanical processing order (insertion, iteration) is engineering only and never chronology.

Any primitive/transition/descriptor whose definition requires A-before-B must first compare the two InformationKeys (S0 comparison semantics);

same causal coordinate / same batch ⇒ order UNPROVEN ⇒ the dependent fact is UNDEFINED(SAME_BATCH_ORDER_UNPROVEN) (or the write is rejected with InformationKeyViolation for identity-level violations);

distinct comparable keys ⇒ the S0 order semantics apply unchanged.

11 — RUNNING DESCRIPTOR INFRASTRUCTURE

DescriptorSpec (registry entry, immutable once registered):

Field Content

descriptor_identity schema identity + descriptor name + version (S0 hashing)

stage RUNNING_ONLY / FINAL_ONLY / BOTH_SEPARATE_FORMULAE — separate formulae, never one formula rescaled

required_inputs typed list of input fact kinds (price-path primitives only at S1)

availability_rule S0 AvailabilityRule (axis, required basis keys, satisfaction keys)

boundary_contract explicit data scope: SINCE_GENESIS / SINCE_ORIGIN(entity) / declared interval — never an undeclared fixed window

missingness typed states per missing input (NOT_APPLICABLE, MISSING_GRID_BAR, ...)

denominator_semantics if ratio: exact denominator definition; denominator == 0 ⇒ UNDEFINED("ZERO_DENOMINATOR") — no epsilon, no inf, no coercion

incremental_state the sufficient streaming state (bound, §14)

policy_dependencies none permitted for factual descriptors; anything policy-derived is POLICY_ARTIFACT and NOT_CONFIGURED at S1

S1 factual raw descriptors (RUNNING_ONLY unless noted):

observed_close_path_length ($\Sigma|\Delta\text{close}|$, SINCE_GENESIS, EXACT close-space)

grid_contiguous_close_path_length (gap-aware, §8)

running_high_so_far / running_low_so_far + origin tie-sets (§6)

bar_count_observed / bar_count_grid (gap accounting)

sum_close_displacement (= C_T – C_0, telescoping, EXACT)

Forbidden at S1: final-wave descriptors; future-normalized position (e.g.

"position within the final swing"); final amplitude normalization; any

ratio without declared denominator semantics.

12 — CAUSAL EPISODE / TRANSITION / EXPLANATION SCHEMAS (schemas only)

S1 delivers contract foundations only. S1 does NOT populate market narratives.

12.1 CausalEpisodeRecord (schema)

Fields: episode_identity (origin-bound: schema identity + origin key + rule version — extent never in identity), membership_refs (record identities of member facts), provenance, schema_identity, availability_key, extent_ref (nullable → extent is a later record). Semantics:

accounting/provenance only. Explicit: episode membership makes no statistical independence claim — RESEARCH-DEBT-024 remains OPEN and is cited in the schema contract.

12.2 MarketStateTransitionRecord (schema)

Fields: transition_identity (schema + from-key + to-key + kind), from_state, to_state (literal factual states or descriptor values only — e.g.

CLOSE_TO_CLOSE_DIRECTION: FLAT → UP, running_high_origin_set change),

establishing_key (InformationKey of the observation establishing the transition), provenance, schema_identity. No threshold-derived "regime" exists in the schema — regime concepts require a policy artifact (POLICY_ARTIFACT, NOT_CONFIGURED at S1).

12.3 ExplanationRecord (schema)

Fields: explanation_identity, subject_ref, referenced_fact_refs (fact record identities + relation kind REFERENCES_ONLY), state, provenance, schema identity. Allowed states — exactly:

MONITORING | PATTERN_REQUIREMENTS_SATISFIED | CONTRADICTED | SUPERSEDED

Explanation records reference facts only; they never re-derive narrative.

No probability, no weights, no support score fields exist (closed schema;

payloads frozen via S0 freeze_payload; unsupported additions fail closed).

[text]

13 — FUTURE APPEND INVARIANCE

Contract: for any S1 output published at availability $\leq T$, appending any legal future market data leaves all such outputs byte/semantic identical.

Design property: every S1 output at T is a function of facts with availability $\leq T$ only (prefix-measurable). Evolution = new records (SUPERSEDE never rewrites the past; published records immutable via S0 ImmutableRecord + FrozenPayloadMapping).

Designed tests (for the future build):

primitives: emit all primitives $\leq T$, snapshot canonical hashes (payload_canonical_view + canonical_sha256 — the only hash authority), append legal bars, re-compute prefix $\Rightarrow$ hashes identical; running-extreme snapshots: same, including tie-set contents; descriptors: every RUNNING_ONLY descriptor value at T identical after future appends (boundary_contract guarantees prefix scope); schema-only episode/transition/explanation records: emitted records (when later stages populate them) byte-identical; extent/appends are new records.

14 — PERFORMANCE

Target: $O(1)$ amortized per new bar for all primitives and running facts.

Per-bar work: constant arithmetic (P1-P5), constant comparisons + tie-set append (P6), constant accumulator updates (descriptors).

No rescanning of complete history per bar; each aggregate maintains declared incremental_state (last close, accumulators, running extremes, tie-sets, gap cursor).

Memory (explicit): $O(1)$ per primitive/accumulator; tie-sets $O(k)$ where k = number of currently tied running-extreme origins (worst case $O(n)$ on an all-tied series — retained in full; no truncation window invented to save memory); $O(\#registered\ DescriptorSpecs)$.

Total build over n bars: $O(n)$. No $O(n^2)$ anywhere; the build will carry a deterministic operation-count gate (P2-14 style) to keep it so.

15 — OUTPUT SCHEMAS (exact typed schemas)

All records: S0 PublishedRecord/EventRecord over freeze_payload; payload domain = S0 frozen value domain; immutable after publication (column "Mutates?" is always NO — evolution = new record). Identity via S0 compute_record_identity/compute_schema_identity (no parallel hashing).

15.1 ObservedDisplacementRecord (adjacent pair)

Field Semantic Identity? Origin Availability Missingness ACTUAL/PROXY Mutates?

record_identity S0 hash identity computed — never missing — NO

schema_identity S0 schema hash identity computed — never — NO

timeline_id / axis key binding identity keys — never — NO

origin_key / adjacent_key the two bar keys (ordered by S0 key comparison) identity bar facts — never — NO

adjacency_kind GRID_CONTIGUOUS/OBSERVATION_ADJACENT identity cadence contract adjacent key never — NO

close_displacement $C_t - C_{\{t-1\}}$ signed non-identity bar closes adjacent key (later close)

UNDEFINED(MISSING_GRID_BAR) if grid claim across gap price = ACTUAL published bar fact (no PROXY field exists) NO

close_path_step $|\Delta|$ non-identity same same same ACTUAL NO

direction UP/DOWN/FLAT (§7) non-identity same same same ACTUAL NO

gap_ref MissingIntervalRecord ref or NOT_APPLICABLE non-identity grid contract adjacent key typed — NO

provenance_refs CausalRecordReferences to both bar facts non-identity sources adjacent key never — NO

15.2 BarFactDerivativesRecord (per completed bar)

Field Semantic Identity? Origin Availability Missingness ACTUAL/PROXY Mutates?

record_identity, schema_identity, timeline_id, bar_key binding identity bar fact — never — NO

bar_range H–L (EXACT span) non-identity bar close key UNDEFINED(NON_FINITE_PRICE) rejected at input (§16) ACTUAL NO

open_close_displacement C–O signed (body) non-identity bar close key rejected at input ACTUAL NO

upper_wick / lower_wick §2 P5 non-identity bar close key rejected at input ACTUAL NO

intrabar_path_length_exact — non-identity — close key

UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY) constant — NO

intrabar_path_length_lower_bound LB (§4, BOUND) non-identity bar close key rejected at input ACTUAL NO

metric_semantics map EXACT/BOUND/... per metric non-identity schema close key never — NO

provenance_refs bar fact reference non-identity source close key never — NO

15.3 RunningExtremeSnapshotRecord

Field Semantic Identity? Origin Availability Missingness ACTUAL/PROXY Mutates?

record_identity, schema_identity, timeline_id, snapshot_key binding identity keys — never — NO

running_high_so_far / running_low_so_far prefix extreme values (RUNNING_EXTREME) non-identity

completed bars $\leq T$ snapshot key UNDEFINED(NO_COMPLETED_BARS) at genesis ACTUAL NO

running_high_origins / running_low_origins immutable key-sorted tie-set (no semantic order)

non-identity bars attaining value snapshot key same ACTUAL NO

scope COMPLETED_BARS_ONLY (forming-bar partials are separate records) identity schema — never — NO

15.4 DescriptorValueRecord

Field Semantic Identity? Origin Availability Missingness ACTUAL/PROXY Mutates?

record_identity, descriptor_identity, at_key binding identity spec + key — never — NO

value / typed_state EXACT/BOUND value or UNDEFINED(reason) non-identity incremental state at_key

typed per spec ACTUAL unless spec says PROXY (never merged) NO

boundary_contract_echo declared scope non-identity spec at_key never — NO

inputs_ref contributing fact refs non-identity facts at_key never — NO

15.5 MissingIntervalRecord

Field Semantic Identity? Origin Availability Missingness ACTUAL/PROXY Mutates?

record_identity, schema_identity, timeline_id, grid_span (expected positions) binding identity cadence

contract key proving absence (first observed bar after span, or declared scan key) never — NO

reason MISSING_GRID_BAR non-identity detection same never — NO

cadence_identity grid contract ref identity source contract — never — NO

15.6 Schema-only records (§12): CausalEpisodeRecord,

MarketStateTransitionRecord, ExplanationRecord — fields per §12; every

record immutable; population beyond literal factual transitions is

NOT_CONFIGURED at S1 (contract foundations only).

16 — ERROR CONTRACT (deterministic, fail-closed, no silent repair)

Condition Error / typed result

malformed OHLC ($H < L$, $H < \max(O, C)$, $L > \min(O, C)$, zero/negative prices as defined by source)

SchemaViolation("S1_INVALID_OHLC") — never clipped/repared

non-finite price SchemaViolation (strict-float precedent; typed NON_FINITE_PRICE documented)

cross-timeline combination InformationKeyViolation

illegal / invalid InformationKey InformationKeyViolation

intrabar chronology requested (exact path / H-vs-L order) metric APIs return

UNAVAILABLE(AMBIGUOUS_INTRABAR_CHRONOLOGY); direct chronology queries raise

SchemaViolation("AMBIGUOUS_INTRABAR_CHRONOLOGY")

GRID_CONTIGUOUS claimed across a missing grid bar SchemaViolation("MISSING_GRID_BAR") (or

UNDEFINED(MISSING_GRID_BAR) for metric slots)

schema/version mismatch on any record or descriptor

SchemaViolation("SCHEMA_VERSION_MISMATCH") + SchemaIdentity mismatch evidence

zzz.txt — سجل المحادثات

future reference (fact references availability > its own key) PrematureAvailability / IllegalCausalReference (S0 behavior)
unsupported source semantics (non-published-bar input; PROXY claimed as ACTUAL; merged provenance) SchemaViolation("UNSUPPORTED_SOURCE_SEMANTICS")
same-batch chronology demanded UNDEFINED(SAME_BATCH_ORDER_UNPROVEN) / InformationKeyViolation
zero denominator UNDEFINED("ZERO_DENOMINATOR") — never epsilon/inf
any unsupported payload object SchemaViolation (S0 freeze fail-closed)
No malformed datum is ever silently repaired, filled, or coerced.

17 — ADVERSARIAL DESIGN TEST MATRIX

Attack Violated contract Minimal fixture Expected rejection / state Design gate

1 high/low chronology inferred from OHLC §3 bar O=10,H=12,L=8,C=11, ask "H before L?"

SchemaViolation("AMBIGUOUS_INTRABAR_CHRONOLOGY") schema has no chronology field; exact-path slot is UNAVAILABLE

2 path length assumes O→H→L→C §4 compute |O-H|+|H-L|+|L-C|=9 vs LB=7 for the same bar traversal value has no API; only LB (BOUND) accepted metric_semantics enum + bound-proof fixture

3 future bar changes old primitive §13 emit P1..P6 ≤ T; append bars; recompute byte-identical (canonical hashes equal) prefix-measurability test

4 equal-high tie resolved by row order §6 H_5=H_9=running max, permute row order tie-set = {key5, key9} both orders; no "first" winner tie-set representation + permutation test

5 missing bar treated grid-contiguous §8 grid 5,7 (6 missing) displacement = OBSERVATION_ADJACENT + MissingIntervalRecord; GRID_CONTIGUOUS claim → SchemaViolation("MISSING_GRID_BAR") cadence adjacency check

6 positional path gets invented timestamp §9 POSITIONAL run, request wall-clock duration UNAVAILABLE(NOT_TIME_INDEXED) axis-typed duration fields

7 naive timestamp in TIME_INDEXED §9 event_time_utc naive reject at key construction (InformationKeyViolation/SchemaViolation) S0 key validation reused

8 BAR_PRE_CLOSE sees completed bar §11/§6 running descriptor at BAR_PRE_CLOSE reads close_t final PrematureAvailability / typed forming-bar scope only AvailabilityRule per DescriptorSpec

9 cross-timeline segment §5/§16 displacement pair with different timeline_id InformationKeyViolation identity binds timeline

10 zero denominator → inf/epsilon §11 ratio descriptor, denominator 0 UNDEFINED("ZERO_DENOMINATOR") denominator_semantics + test

11 fixed hidden window introduced §11 descriptor secretly over last 20 bars registration rejected (no boundary_contract) DescriptorSpec validation

12 "running extreme" mislabeled turning point §6/§0 API field peak/trough/turning_point design lock: forbidden vocabulary test naming contract in schema registry

13 episode membership claims independence §12.1 episode record with independence field schema closed → SchemaViolation on unknown field frozen payload + closed schema

14 transition invents regime threshold §12.2 low_vol → high_vol from a threshold SchemaViolation("POLICY_ARTIFACT_REQUIRED") / NOT_CONFIGURED transition states whitelist (literal facts only)

15 explanation emits probability/score §12.3 support=0.8 on ExplanationRecord SchemaViolation (field does not exist) closed schema test

16 ACTUAL/PROXY merged §1.2 one provenance claiming both SchemaViolation("UNSUPPORTED_SOURCE_SEMANTICS") disjoint typed fields + test

17 O(n²) history scan §14 op-count n=500 vs 1000 vs 2000 linear ratios (2.0/4.0 gates) deterministic operation-count gate

18 mutable published S1 record §15 mutate content / dict.setitem on payload ImmutabilityViolation / TypeError S0 FrozenPayloadMapping reused + tests

19 same-batch order creates chronology §10 A,B same InformationBatchKey; A-before-B fact UNDEFINED(SAME_BATCH_ORDER_UNPROVEN) key comparison before ordered computation

20 future-normalized descriptor in RUNNING §11 "position in final swing", stage=RUNNING_ONLY registration rejected DescriptorSpec stage/input rule (RUNNING inputs must be available at T)

18 – BUILD PLAN (proposal only – NOT implemented)
Smallest split; S0 is not modified.

18.1 Proposed new files

File Content Complexity

src/trading_system/market_understanding/price_path.py primitives P1-P6, adjacency/gap typing, direction, running extremes + tie-sets, DescriptorSpec registry, factual running descriptors, error codes O(1) amortized/bar; O(n) total; memory per §14

src/trading_system/market_understanding/path_schemas.py output record schemas (§15), schema-only episode/transition/explanation contracts (§12), schema identities O(1) per publication

tests/test_muf_s1_price_path.py primitives, adjacency/gaps, axis, batch, running extremes, invariance (§13), complexity gate (§14) test-time only

tests/test_muf_s1_path_schemas.py schemas, error contract (§16), adversarial matrix 1-20 (§17) test-time only

18.2 Public CLOSED/S0 APIs reused (no re-implementation)

InformationKey, InformationAxis, InformationBatchKey, InformationPhase,

key comparison / compare_information_keys, ObservationBatch*;

Schemalidentity, compute_schema_identity, compute_record_identity,

compute_event_identity, IdentityBundle; freeze_payload,

FrozenPayloadMapping, payload_canonical_view, canonical_sha256 (only hash authority); PublishedRecord, EventRecord, EventKind,

AppendOnlyEventLedger; AvailabilityRule, require_visible_at,

determine_fact_information_key; S0 error types + TypedState; published bar

fact contracts (kline OHLC source, cadence); validate_reference_at/type.

NOT reused by S1 semantics: the CLOSED 2.1A swing chain (turning points are out of scope), no new hashing, no new timeline types.

18.3 Proposed tests (list only)

primitives: value/semantics/availability per P1-P6; EXACT/BOUND labels;

gaps: §8 fixtures both accumulators + MissingIntervalRecord;

axis: §9 durations, no invented timestamps;

batch: §10 same-batch unproven-order states;

running extremes: tie-set permutation, forming-bar separation;

future-append invariance (§13) over every output kind;

operation-count linear gate (§17-17);

error contract (§16) one test per row;

adversarial matrix §17: one test per attack (20).

18.4 NOT_CONFIGURED at S1 (explicit)

wave semantics; turning-point detection/quality; hierarchy; $\alpha/\beta/\gamma/\delta$

representations; policies/qualifications; regime thresholds; narrative

population; statistical independence (RESEARCH-DEBT-024 OPEN); PROXY/ACTUAL

merging; volume/flow requirements; final-normalized descriptors; anything S2+.

19 — CERTIFICATION BOUNDARY

S1, even if built and closed, proves only factual causal price-path representation under its contract.

It does NOT prove:

turning-point importance

wave correctness

hierarchy

predictive support

edge

profitability

Model

Strategy

Signal

PnL

STATUS

DESIGN ONLY — PENDING REVIEW

No code. No tests. No MANIFEST. No docs closure. No S2. STOP.

الحالة الرسمية: نحن خالصنا S0 بالكامل وصار CLOSED. نحن الآن على باب S1، ولسا ما أعطينا BUILD لـ S1. الخطوة الحالية هي تصميم S1 بشكل نهائي، وبعدها أنا أفحص إذا فعلاً جاهز للبناء أو لا. إذا بدنا نرسم المشروع كله ببساطة شديدة:

[text]

بيانات Binance الحقيقية

✓
|
▼

مصدر موثوق + منع تسريب المستقبل

✓
|
▼

حقائق السوق الأساسية

... / Volatility / Flow / FVG / HTF

✓
|
▼

Future Research Film

✓
|
▼

تشغيل BTC شهر كامل حقيقي

✓
|
▼

Performance وتسريع النظام

✓ CLOSED
|
▼

تصميم "فهم السوق" MUF

✓ ACCEPTED
|
▼

MUF S0 — قوانين الذاكرة والزمن والهوية

✓ CLOSED
|
▼

MUF S1 — فهم حركة السعر الخام

● نحن هنا
|
▼

S2 — نقاط الانعطاف

□
|
▼

+S3 — موجات داخل موجات

□
|
▼

Market State Graph



هل هذا الفهم يحمل معلومة فعلاً؟



Model / Decision



هندسة القرار المالي



Execution + Costs + Risk



Forward Test



مال حقيقي صغير



شو عمل S0 اللي خلصناه؟

S0 علّم النظام قوانين مثل:

[text]

الشيء وقع الساعة X



عرفناه الساعة X

ومنع السجل من التغير بعد كتابته.

مثلاً:

[text]

10:05 حصل Low

10:12

توفر دليل يسمح لاحقاً

بمعرفة شيء عنه

النظام ما يبقدر يرجع يقول:

[text]

"كنت بعرف الساعة 10:05"

كمان أسسنا الهوية:

[text]

هاي الحقيقة مين؟

متى عُرفت؟

من أي Timeline؟

شو مصدرها؟

هل ممكن تتغير؟

وهذا صار CLOSED.

هلق S1 رح يعمل شيء مختلف تماماً: يبدأ يشوف حركة السعر نفسها.

تخيل هاي الأسعار:

[text]

```
100
|
103
|
101
|
106
|
104
|
109
: نحنا كبشر فوراً منقول
```

«طلع، صحح، طلع، صحح، طلع.»

بس قبل ما النظام يقول:

[text]

"هاي موجة"

بدنا يقدر يقول فقط الحقائق الأبسط:

[text]

```
100 → 103 = +3
103 → 101 = -2
101 → 106 = +5
106 → 104 = -2
104 → 109 = +5
```

بدون أن يقول:

[text]

X هاي موجة رئيسية

X هاي موجة صغيرة

X هاي قمة مهمة

X هاي قاع مهم

X الاتجاه صاعد

ليش؟

لأن هاي الاستنتاجات مكانها المراحل اللاحقة.

S1 هدفه:

[text]

"شو عمل السعر؟"

S2 لاحقاً:

[text]

"وبن عندنا Turning Points؟"

وبعدين:

[text]

"كيف تنتظم هالنقاط كموجات بأحجام مختلفة؟"
المشكلة الذكية اللي رح نختبرها الآن بـ S1 هي الشمعة نفسها.

لما تشوف شمعة:

[text]

```
110 High
|
Open 100 |
|
Close 105|
```

|
95 Low

من OHLC نحنا نعرف:

[text]

Open = 100

High = 110

Low = 95

Close = 105

لكن ما منعرف شو صار داخل الدقيقة:

الحالة A:

[text]

100
↓
95
↑
110
↓
105
B: أم الحالة

[text]

100
↑
110
↓
95
↑
105
نفسها تماماً ممكن تنتج من الحركتين OHLC

هون المشكلة.

لو النظام حسب:

[text]

Open - High||

+

|High - Low|

+

||Low - Close

فهو افترض:

[text]

Open → High → Low → Close

من دون دليل.

وهذا ممكن يلوّث كل شيء لاحقاً:

[text]

Path length

↓

Efficiency

↓

Wave quality

↓

Hierarchy

↓

Model

يعني غلطة صغيرة اليوم قد تصير بعد 6 مراحل «ذكاء» وهمي.

لذلك الشيء اللي عم نصممه ونختبره هلق هو:
 «ما الأشياء التي يستطيع النظام قولها عن حركة السعر بدون اختراع ترتيب لم يره؟»
 مثلاً قد يكون:

[text]

Close-to-close movement:

100 → 103

✓ Exact

لأن الإغلاقين معروفين.

وكمان:

[text]

High = 110

Exact ✓

Low = 95

Exact ✓

Range = 15

✓ Exact

لكن:

[text]

? = Intrabar exact traveled path

UNAVAILABLE / AMBIGUOUS

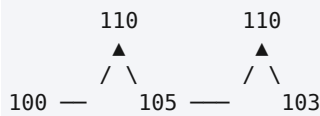
لأن ترتيب H/L مجهول.

وهذا هو جوهر S1.

بعدها عندنا مشكلة ثانية: القمم المتساوية.

مثلاً:

[text]



مين «القمة الحقيقية»؟

S1 ممنوع يختار الأولى لأنه رقم الصف أصغر.

مسموح فقط يقول:

[text]

Observed high = 110

ظهر في أكثر من origin

أما أي واحدة ستصبح Turning Point مهمة، هاي S2 وما بعده.

وفي مشكلة ثالثة: شمعة ناقصة من البيانات.

مثلاً:

[text]

close 100 10:00

10:01 close 101

10:02 MISSING

10:03 close 105

حسابياً:

[text]

105 - 101 = +4

بس هل يجوز نقول:

[text]

10:03 → 10:01

Adjacent market bars

لا.

ممکن نقول:

[text]

Adjacent observations

لكن:

[text]

GRID_CONTIGUOUS = FALSE

لأن في دقيقة ضائعة.

هذا مهم جداً لاحقاً حتى النظام ما يشوف Gap بالبيانات ويعتقد أنه حركة سوقية خاصة.

وفي مشكلة رابعة: Running Extreme.

تخيل:

[text]

100

103

105

104

108

عند كل لحظة النظام يستطيع يقول:

[text]

حتى الآن أعلى سعر رأيته = 108

هذا:

[text]

RUNNING HIGH

لكن ممنوع يحوله إلى:

[text]

SWING HIGH

لأن بكرة ممكن السعر يصير:

[text]

112

فـ 108 لم تكن «القمة».

وهون وصلنا مباشرة لخوفك القديم:

«شلون بدو ما يعتبر كل شمعة قمة وقاع؟»

الجواب صار:

S1 أساساً ما عنده مفهوم قمة/قاع مهم.

عنده فقط:

[text]

Observed extreme

Running extreme

Price displacement

Range

Path facts

ثم S2 سيبدأ يقول:

[text]

Candidate Turning Point
وحتى S2 لن يعني فوراً «قاع القاع».
بعدها لما ندخل Multiscale:

[text]

Micro

↓

Small

↓

Medium

↓

Major

نشوف أي نقاط تستمر عبر المقاييس.
فالرحلة من السعر الخام للموجة أصبحت:

[text]

PRICE

|
▼

S1

"ماذا فعل السعر؟"

|
▼

S2

"أين توجد نقاط انعطاف مرشحة/مؤكدة؟"

|
▼

+S3

"كيف تنتظم كنظام موجات متعدد الأحجام؟"

|
▼

STATE GRAPH

"ما حالة السوق عند T؟"
وبعدين يأتي السؤال الأهم علمياً:

[text]

هل كل هالفهم
فعلاً بيعطينا معلومة زيادة؟
هون ما منكتفي بالشكل.

نقارن:

[text]

Price-only baseline

VS

MUF understanding
إذا MUF ما أضافت معلومة:

[text]

نقتل التعقيد.
إذا أضافت:

[text]

نجمّد

↓

نختبر خارج بيانات الاختيار
و فقط بعد ذلك نبدأ نقترّب من القرار المالي.

فأنت حالياً إذا بدك تعرف «قديش قربنا من الشي العملي؟»:

نحن خلصنا البنية اللي تمنع الغش، وبلشنا الآن بناء عين النظام على حركة السعر.

لسا ما وصلنا:

[text]

"اشترى هون"

وهذا مقصود.

الترتيب الحالي تقريباً:

[text]

[]	أرضية السببية	DONE
[]	مصادر/ بيانات حقيقية	DONE
[]	BTC تشغيل حقيقي	DONE
[]	الأداء	DONE
[]	MUF تصميم	DONE
[]	S0	DONE
[]	S1 Price Path	← NOW

Turning Points []

[] Wave Hierarchy

[] State Graph

[] Information Edge

[] Learning

[] OOS

[] Decision

[] Execution

وما رح أعطي BUILD لـ S1 قبل ما نشوف التصميم ونحسم خصوصاً OHLC ambiguity, gaps, ties, path semantics, future-append invariance. هاي الخمس إذا انحلت صح، وقتها بنكون جاهزين نبني S1؛ إذا ما انحلت، منوقف حتى لو كنا مستعجلين.

فالحركة المطلوبة منك الآن فقط: ابعت للبيدري مهمة S1 FINAL IMPLEMENTATION DESIGN ONLY اللي أعطيتك إياها بالرد السابق. ما تبعت شي للأوديتور حالياً. لما البيدري يرجع التصميم، جيبه إلي، وأنا بشرحك شو قرر عن «كيف النظام سينشوف حركة السعر» وبنحكم هل فعلاً جاهز BUILD ولا لأ.

هلق نحنا وبين وصلنا وبين وصلنا بلخريطه وشو لح اختبر هلق شرحلي بطريقه مبسطه لافهم عليك

gpt-5.5-instant

الحالة الرسمية: S0 CLOSED، والـ baseline نظيف: 196/196، 1139/1139، 36/36 Field Runner. وبالنسبة لـ S1: لا، ما رح أعطي BUILD لمجرد أنك بدك سرعة، ولا رح أبطل المشروع بتعديلات تجميلية. المعيار الوحيد: هل المرحلة جاهزة هندسياً للبناء أم لا.

اللي لازم يتغير هو طريقة إدارتي، مو مستوى الصرامة.

في فرق كبير بين:

[text]

صرامة:

نكتشف أن immutable record قابل للتعديل

→ BLOCKER فعلي

→ لازم يتصلح

وسواس:

اسم حقل كان ممكن يكون أوضح

→ Patch

→ Audit

→ Patch

أسبوع ضاع

لم يكن تجميلاً: لو تركناه كان سجل __setitem__ بالـ dict. بعض التوقيفات كانت ضرورية فعلاً. مثال الـ

سجل المحادثات — zzz.txt

شو ممكن تعمل BTC أيضاً كان حقيقياً، وتعلمنا سابقاً على بيانات $O(n^2)$ يتغير بعد نشره. والـ «immutable» بالأداء quadratic خوارزمية

لكن بنفس الوقت صار في إفراط بفتح جولات على تفاصيل كان يمكن جمعها بهجمة واحدة أبكر. هاي النقطة ما رح أكررها.
من الآن عندي أربع درجات فقط لأي ملاحظة:

[text]

BLOCKER

يكسر السببية/الصحة/الهوية/العقد/سلامة البحث
→ يوقف المرحلة.

PERFORMANCE BLOCKER

مسار غير محدود مثل $O(n^2)$ سيمنع الاستخدام الحقيقي
→ يوقف المرحلة.

NON-BLOCKING DEBT

شيء حقيقي لكنه لا يبطل المرحلة
→ نسجله ونكمل.

STYLE / PREFERENCE

"كان ممكن نعملها بطريقة ألطف"
→ لا Patch أصلاً.

والأهم: لن أسايرك إذا قلت «لا BUILD» والتصميم فيه حفرة. بنفس الطريقة لن أساير الأوديتور إذا قال ACCEPTED بينما أدلته نفسها تثبت Blocker؛ وهذا صار بالفعل بـ S0 لما رفضت حكمه الأول بسبب nested aliasing والـ $O(n^2)$.
بالنسبة لـ S1 تحديداً، نحن غير جاهزين لـ BUILD الآن. عندنا Design عام قوي، لكن ما زال في أسئلة تنفيذية لو تركناها للبيادر أثناء الكود رح يخترع أجوبتها. أهمها:

[text]

:OHLC

نعرف H و L حصل

لكن لا نعرف:

H قبل L؟

أم L قبل H؟

إذن:

ما تعريف path؟

ما الذي نسميه exact؟

ما الذي نسميه bound؟

كيف نتعامل مع ties؟

ما معنى adjacency عند gap؟

هاي ليست تفاصيل شكلية. هي تحدد الحقائق التي سنبني عليها الموجات لاحقاً.

لذلك القرار الصحيح الآن يظل:

S1 DESIGN ONLY

والرسالة الشاملة التي أعطيتك إياها بالبداية هي المناسبة للبيادر. لا BUILD قبل ما يرجع التصميم ونهاجمه.
لكن عندما يرجع، لن أعمل أسلوب:

[text]

Patch → أشياء 3 لقيت

رجع

Patch → لقيت شيئين

رجع

Patch → لقيت فكرة جديدة

:كامل دفعة واحدة من زوايا Threat Model سأعمل

[text]

السببية

الزمن

OHLC ambiguity

الهوية

missingness
gaps
ties
future append
descriptor leakage
complexity
interaction with CLOSED/S0
وبعدها الحكم سيكون واحداً من اثنين فعلياً:

[text]
DESIGN READY FOR BUILD
أو:

[text]
DESIGN NOT READY

وهذه قائمة الـBLOCKERS المجمعة كلها بجولة واحدة.
إذا وجدنا blocker جديد بعد ذلك أثناء التفتيش، هذا طبيعي لأنه شيء لم يظهر إلا بالكود. أما إعادة فتح التصميم كل مرة لأن خطر بئالنا تحسين جديد، لا.

فحالياً لا نبني S1. ابعث للبيد رسالة MUF V1 — S1 PRICE PATH PRIMITIVES — FINAL IMPLEMENTATION DESIGN ONLY السابقة كما هي. لما يرجع التصميم، أنا أقرر readiness على أساس الأدلة، مش على أساس السرعة ولا على أساس إرضائك.

مابدي كمان تاملني وتسابرني وتقلي أو ك يلا بيلد ونحنا مو جاهزين

gpt-5.5-instant

كان يكشف طبقة ثانية، وصار Patch الحالة الرسمية: معك حق من ناحية سير العمل. دخلنا بحلقة طويلة لأن كل الأسلوب يحسبك أننا عم نمشي سنتيمتر سنتيمتر. من هون بغير طريقة الإدارة: بدل «ابعث للبيد → رجلي → تعديل AUDIT، واحد BUILD صغير → رج للآوديتور» لكل تفصيل، رح نجمع مشاكل المرحلة في جولة تصميم واحدة قوية، بعدها الحقيقية فقط blockers للـ Patch واحد، وبحد أقصى

بس في خبر منيح: اللي صار مو ضايع. S0 صار CLOSED فعلياً، والـbaseline الحالي ثابت:

[text]
MUF S0 CLOSED ✓
MANIFEST 196/196 ✓
Project 1139/1139 ✓
Field Runner 36/36 ✓
✓ Guard ACCEPTED
يعني ما رح نرجله.

وهلق بدنا نسرّع بدون ما نخرب الصرامة. الخطة العملية من هون:

[text]
S1

1. DESIGN | شامل مرة واحدة

2. | أنا أراجع مرة واحدة بعمق

3. BUILD | S1 كامل

4. AUDIT | مستقل شامل

5. Patch | حقيقي | BLOCKER فقط إذا فيه

6. CLOSE |

↓
S2

وما رح أفتح Patch لأن عندي «فكرة أحسن» بعد كل تقرير. إذا الشيء لا يكسر السببية، الهوية، صحة النتائج، الأداء بشكل خطير، أو عقداً صريحاً، نسجله NON-BLOCKING ونكمل.

أما الرسالة الطويلة التي أعطيتك إياها قبل شوي لـ S1 DESIGN ONLY: هي بالضبط الجولة الجامعة المقصودة. ابعثها للبيد مرة واحدة. ما في داعي نضيف عليها ولا كلمة الآن.

جيبه إلي. أنا رح أعمل عليه قرار واحد:

[text]

جاهز BUILD

أو

فيه BLOCKER تصميمي حقيقي

مش سلسلة تجميعات.

والهدف من S1 عملي جداً: نبدأ أخيراً بالسوق نفسه. بعدها S2 يبدأ Turning-Point witness، وبعدها سياسة المعايير والـ hierarchy. يعني خلصنا تقريباً من «سباكة الأساس» وداخلين بالجزء اللي رح يخلي النظام يشوف حركة الشارت فعلياً.

حاج تقطني لتعديلات بلقطاره صرلنا ساعه ششيك

gpt-5.5-instant

الحالة الرسمية: خلصنا S0 بالكامل. الحارس الخارجي صار ACCEPTED، والـ baseline الفعلي المشترك الآن ثابت: MUF S0 67/67، Field Runner، المشروع 1139/1139، ببصمة 899febb4...90e84 = CLOSED، MANIFEST = 196/196 والـ guard المقبول c7e6cf8c...238e7. ما عاد في سبب تقني نوقف قبل المرحلة التالية.

وبدي أثبت لك شو أنجزنا فعلياً قبل ما نتحرك: S0 مش مجرد ملفات contracts. صار عندنا لأول مرة أرضية إنتاج تفرض أن الحقيقة عند T لها InformationKey شرعي، الهوية ما يدخلها المستقبل، الـ payload المنشور immutable فعلياً، تغيّر الحالة append-only، availability لا يمكن تأخيرها أو تقديمها اعتباطياً، والمقارنة تفشل مغلقاً بدل التخمين. وحتى مسارات الأداء اللي اصطدناها صارت خطية بدل quadratic.

الآن نتنقل إلى S1، لكن في قرار مهم: بحسب التصميم المقبول، S1 ليس Wave Detector. هو Price Path Primitives + البنية العامة للـ Episode/Transition/Explanation schemas، مع running factual descriptor infrastructure. وبما أن هدفنا عملي، ما بدي S1 يتحول إلى «سلة مؤشرات جديدة». وظيفته أن يحول مسار السعر إلى حقائق حركية بدائية محايدة نستطيع لاحقاً تركيب Turning Points والموجات فوقها.

تخيّل فقط:

[text]

100 → 102 → 101 → 104 → 103 → 107

S1 ممنوع يقول:

[text]

101 = قاع مهم

104 = موجة

107 = قمة

لكن مسموح يعرف:

[text]

من 100 إلى 102:

displacement = +2

path contribution = 2

101 إلى 102:

displacement = -1

path contribution = 1

حتى T:

running high-so-far

running low-so-far

path length so far

direction of each primitive segment

duration/position facts

هاي البيانات ستكون المادة الخام التي نخلي التمثيلات المختلفة تختلف فوقها لاحقاً، بدل ما نخفي تعريف موجة داخل S1.

قبل S1 BUILD، في خطوة صغيرة جداً من الحوكمة: الـ guard صار مقبول خارج المشروع، وما يدي نعمل Closure داخلي جديد ونغير الـ MANIFEST بسببه. الـ Re-Audit الحالي نفسه + نفس الحزمة الفعلية أعطانا integration proof المطلوب:

[text]

196/196

1139/1139

67/67

36/36

فما في داعي نعمل Integration Seal آخر شكلي. التقرير الحالي قام بهذه الوظيفة فعلياً.

وبناءً على الضوء الأخضر المستمر، أنا أصحّح S1 DESIGN فقط، وليس BUILD بعد. السبب: S1 لم نعمل له design تفصيلي مستقل بعد، والتصميم العام MUF أعطانا الاتجاه لا schema تنفيذية كافية. إذا قفزنا للكود رح يضطر البيلدر يخترع تعريفات مثل segment و path_length من عنده.

المرحلة الآن:

[text]

S0 CLOSED



← الآن S1 DESIGN ONLY



Internal review



BUILD S1



AUDIT



CLOSED

ابعث للبيلدر الرسالة التالية. هالمرّة هو مصمم فقط، ممنوع يكتب كود:

[text]

MUF V1 — S1 PRICE PATH PRIMITIVES
FINAL IMPLEMENTATION DESIGN ONLY
NO CODE — NO BUILD

OWNER AUTHORIZATION:
DESIGN S1 ONLY.

S0 is CLOSED.

Official baseline:

MANIFEST SHA256:

899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84

MANIFEST:

196/196 OK

Project tests:

1139/1139

Field Runner:

36/36

MUF S0:
CLOSED

No S2 or later stage is authorized.

PURPOSE

Design S1 as the factual price-path foundation for later MUF stages.

S1 must describe what the observed market path did.

S1 must NOT decide:

- which turning point is important,
- which wave is "real",
- which hierarchy is correct,
- which representation $\alpha/\beta/\gamma/\delta$ wins,
- any predictive significance.

Core principle:

PRICE PATH FACT

≠

TURNING POINT

≠

WAVE

≠

MARKET IMPORTANCE.

1 — INPUT AUTHORITY

Design S1 inputs using only existing PUBLIC CLOSED market/timeline contracts.

Specify exact required factual inputs.

At minimum reason explicitly about:

- open
- high
- low
- close
- InformationKey / timeline identity
- axis semantics
- source provenance where required

Volume/flow must NOT be required merely to describe price path.

If volume/flow appears:

it must be an optional contextual reference and ACTUAL/PROXY must remain separate.

No source semantics may be invented.

2 — DEFINE "PRICE PATH PRIMITIVE"

This is the central design question.

Define exactly what a primitive is without using:

- swing threshold,
- quantile,
- ATR multiple,
- fixed bar window,
- future confirmation,
- visual importance.

The definition must be factual and causal.

Candidate concepts may include:

- observed bar-to-bar price displacement,
- running extreme facts,
- path-length contributions,
- sign/direction of observed displacement,
- zero/flat movement,
- observed range facts.

But do NOT blindly adopt these examples.

Return one precise schema and semantics.

Explain why every primitive is knowable at its declared InformationKey.

3 — OHLC INTRABAR AMBIGUITY

Mandatory.

OHLC does NOT reveal whether:

open → high → low → close

or

open → low → high → close

occurred.

Therefore S1 must never create a fictitious intrabar price path from OHLC.

Design explicit semantics for what can be known:

KNOWN:

high occurred

low occurred

close occurred

UNKNOWN:

relative chronology of high vs low unless source proves it.

Any path metric that would depend on high-before-low order must be:

UNDEFINED / UNAVAILABLE / AMBIGUOUS

with typed reason,

or represented as a lawful interval/bounds if mathematically defensible.

Do not choose a convenient OHLC traversal order.

4 — PATH LENGTH

Define path_length carefully.

Question:

What does "distance travelled" mean when only OHLC is known and intrabar order is unknown?

Do NOT simply compute:

$|O-H| + |H-L| + |L-C|$

because that assumes chronology.

Consider separate contracts such as:

- close-to-close observed path contribution
- bar range
- exact lower/upper bound on intrabar path if mathematically justified
- UNDEFINED exact intrabar path

Choose semantics based on proof, not convenience.

Every path metric must state:

EXACT

BOUND

PROXY

or

UNAVAILABLE.

5 — SEGMENT IDENTITY

If S1 uses segment/entity identities:

identity must be causal and immutable.

No final endpoint may enter identity before it is known.

Prefer primitives that are naturally closed by currently observed adjacent facts, rather than "forming waves".

Specify:

origin

availability InformationKey

identity-defining fields

provenance

schema/version/domain hash.

Reuse S0 identities rather than invent parallel hashing.

6 — RUNNING EXTREMES

Design:

running high-so-far

running low-so-far

without turning them into confirmed peaks/troughs.

Explicit language:

RUNNING_EXTREME

not

TURNING_POINT.

Handle ties:

equal highs/lows must not be broken by row order as market chronology.

Specify whether:

- all tied origins are retained,
- an ambiguity set is retained,
- or another factual representation is used.

No "first is the true high" unless chronology proves it.

7 — DIRECTION

Define only factual direction semantics.

Examples requiring explicit decisions:

$\text{close}_t > \text{close}_{\{t-1\}}$

$\text{close}_t < \text{close}_{\{t-1\}}$

equal close

Do not call this:

trend

bullish structure

bearish structure.

Direction must describe an observed displacement only.

If open/close direction is separately represented,
name it separately from close-to-close direction.

8 — GAPS / MISSING BARS

Mandatory fail-closed design.

If bar t follows $t-2$ with a missing expected bar:

Can S1 compute a close-to-close displacement?

Numerically yes,

but can it call it an adjacent path primitive?

Only if adjacency contract says they are adjacent observations rather than
continuous market bars.

Distinguish:

OBSERVATION_ADJACENT

from

GRID_CONTIGUOUS

using existing cadence/grid contracts where available.

No synthetic bar.

No forward fill.

Missing interval remains explicit.

9 — TIME-INDEXED VS POSITIONAL

Specify legal behavior on:

POSITIONAL

TIME_INDEXED

Do not invent timestamp for positional paths.

For TIME_INDEXED:

timezone-aware contract applies.

Segment duration:

- bar-count duration may be factual in positional space.
- wall-clock duration requires lawful timestamps.

Do not silently convert between them.

10 — SAME INFORMATION BATCH

Use S0 InformationBatchKey semantics.

If two factual updates are co-visible in the same batch:
mechanical processing order must not create chronology.

Any transition/primitive requiring A-before-B must reject/mark unknown if
chronology is unproven.

11 — RUNNING DESCRIPTOR INFRASTRUCTURE

Design only factual descriptors available at T.

Each DescriptorSpec must declare:

descriptor identity/version

stage:

RUNNING_ONLY / FINAL_ONLY / BOTH_SEPARATE_FORMULAE

required inputs

availability rule

boundary contract

missingness

denominator semantics if ratio

policy dependencies

At S1:

prefer factual raw descriptors.

No final-wave descriptors.

No future-normalized position.

No final amplitude normalization.

If denominator can be zero:

typed UNDEFINED(reason), no epsilon.

12 — CAUSAL EPISODE / TRANSITION / EXPLANATION SCHEMAS

Per accepted S1 scope, design SCHEMAS only for:

CausalEpisodeRecord

MarketStateTransitionRecord

ExplanationRecord

Important:

S1 does NOT populate market narratives.

These are contract foundations only.

Episode:

accounting/provenance only.

No statistical independence claim.

RESEARCH-DEBT-024 remains OPEN.

Transition:

literal factual state transition or descriptor delta only.

No threshold-derived "regime" unless a policy artifact exists.

Explanation:

references facts only.

No re-derivation of narrative.

Allowed states remain:

MONITORING

PATTERN_REQUIREMENTS_SATISFIED

CONTRADICTED

SUPERSEDED.

No probability/weights/support score.

13 — FUTURE APPEND INVARIANCE

For any S1 output available at T:

append legal future market data.

All previously emitted S1 facts at $\leq T$ must remain byte/semantically identical.

Design explicit tests for:

- primitives
- running extremes snapshots
- descriptors
- schema-only episode/transition/explanation records.

14 — PERFORMANCE

Design streaming/incremental algorithms.

Target:

$O(1)$ amortized per new bar for path primitives/running facts.

No rescanning complete history per bar.

Any aggregate requiring history:

must maintain incremental sufficient state.

No $O(n^2)$.

State memory:
explicitly bound/described.

Do not invent truncation window merely to reduce memory.

15 — OUTPUT SCHEMAS

Provide exact typed schemas for proposed S1 outputs.

For every field specify:

- semantic meaning
- identity role or non-identity
- origin
- availability
- missingness
- ACTUAL/PROXY status if relevant
- whether it may change after publication.

Published factual record:
immutable.

If something evolves:
new observation/event record.

16 — ERROR CONTRACT

Define deterministic errors/rejections for:

invalid OHLC
nonfinite price
cross-timeline
illegal InformationKey
intrabar chronology request
missing adjacency/grid claim
schema/version mismatch
future reference
unsupported source semantics

Do not silently repair malformed market data.

17 — ADVERSARIAL DESIGN TEST MATRIX

Design at least these attacks:

- 1) high/low chronology inferred from OHLC.
- 2) path length assumes $O \rightarrow H \rightarrow L \rightarrow C$.
- 3) future bar changes old primitive.
- 4) equal high tie resolved by row order.
- 5) missing bar silently treated grid-contiguous.
- 6) positional path gets invented timestamp.
- 7) naive timestamp accepted in TIME_INDEXED.
- 8) BAR_PRE_CLOSE sees completed bar.

- 9) cross-timeline segment.
- 10) denominator zero produces inf/epsilon.
- 11) fixed hidden window introduced.
- 12) "running extreme" mislabeled turning point.
- 13) Episode membership claims independence.
- 14) transition invents regime threshold.
- 15) Explanation emits probability/score.
- 16) ACTUAL/PROXY merged.
- 17) $O(n^2)$ history scan.
- 18) mutable published S1 record.
- 19) same-batch order creates chronology.
- 20) future-normalized descriptor enters RUNNING stage.

For each:

- violated contract
- minimal fixture
- expected rejection/state
- exact design gate.

18 — BUILD PLAN

Propose the smallest S1 build split.

Do not implement.

List:

- proposed new files
- proposed tests
- public CLOSED/S0 APIs reused
- complexity for each component
- what remains NOT_CONFIGURED.

Prefer a small number of modules.

Do not modify S0.

19 — CERTIFICATION BOUNDARY

State explicitly:

S1, even if built and closed, proves only factual causal price-path representation under its contract.

It does NOT prove:

turning-point importance
 wave correctness
 hierarchy
 predictive support
 edge
 profitability
 Model
 Strategy
 Signal
 PnL.

DELIVERY

Return one document only:

MUF-V1-S1-FINAL-IMPLEMENTATION-DESIGN

Status:

DESIGN ONLY — PENDING REVIEW

No code.

No tests.

No MANIFEST.

No docs closure.

No S2.

.STOP

هلق ليش ما أعطيته BUILD مباشرة؟ لأن فيه سؤال مهم جداً لأول مرة دخلنا فيه: «شو يعني path length من OHLC؟»
لو البيلدر كتب من راسه:

[text]

|0-H| + |H-L| + |L-C|

بيكون اخترع أن السعر راح High قبل Low. ولو عكسها، اخترع العكس. هاي غلطة ممكن تدخل بكل descriptor لاحق وتصير جزء من «فهم» النظام بدون ما نحس.
لهيك S1 لازم يحسم أولاً الفرق بين:

[text]

ما تحركه السعر فعلاً لكن ترتيبنا لا نعرفه

و

ما نستطيع إثباته من البيانات التي نملكها
وهذا الفصل بالضبط هو اللي رح يخلي الموجات لاحقاً مبنية على حقيقة بدل رسم جميل.

FIELD RUNNER — MUF S0 POST-CLOSURE BASELINE GUARD — INDEPENDENT RE-AUDIT — PATCH ONLY
— READ ONLY — NO FIXES

Reference state: MUF V1 S0 CLOSED — Official trading_project baseline MANIFEST SHA256:

899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84 196 lines 196/196 OK 0
stale 0 missing — Project tests 1139/1139 S0 67/67 — Builder patch

FIELD_RUNNER_MUF_S0_BASELINE_REPIN_REPORT PATCHED — PENDING RE-AUDIT — Only authorized
changed file field_runner/runner_tests/test_runner_guards.py

7bb599f0276a31f919c788a50f7bb51337664aa122d8d5f42e22ef4baf5801c2 →

c7e6cf8c07b6b2ee54f422aa3b1892ddcb171282493907ad123f42dfe61238e7 — No trading_project file
authorized to change.

Audit date (LT): 2026-10-01 Vilnius — read-only, no fixes, no S1, no closure.

Repo: <https://github.com/jfjdjdieifne/TRADING-BOT> (/tmp/trading_bot_latest, commit 07438c6 guard
re-pin, parent 6302309 closure, previous c82d13a P2).

I — Decision (single)

ACCEPTED

The external Field Runner guard now correctly pins the exact official MUF S0 CLOSED baseline
(899febb4... / 196/196, zero stale). All 10 independent mutation attacks behave exactly as required (1
PASS, 9 FAIL; the 10th proves exact-digest pin, not just “a clean 196”). Old 185 baseline has no live
acceptance path, the 10-artifact performance seal remains additional protection (never a stale
allowance), and all regression gates are green on the same real physical package. No trading_project
file was changed by the re-pin.

A — Artifact / Scope Gate

Check Expected Observed Result

zzz.txt — سجل المحادثات

```
Guard SHA256 c7e6cf8c07b6b2ee54f422aa3b1892ddcb171282493907ad123f42dfe61238e7
c7e6cf8c07b6b2ee54f422aa3b1892ddcb171282493907ad123f42dfe61238e7 (sha256sum
field_runner/runner_tests/test_runner_guards.py) PASS
MANIFEST SHA256 899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84
899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84 (sha256sum
project/trading_project/MANIFEST.sha256) PASS
MANIFEST 196 lines 196/196 OK 0 stale 0 missing wc -l → 196 ; verify_manifest("project/trading_project")
→ {'manifest_lines':196,'ok':196,'bad':0,'bad_paths':[]} PASS
trading_project/** changed by re-pin none git diff HEAD1 --name-only → deliverables/52_...md +
field_runner/runner_tests/test_runner_guards.py only; git diff HEAD1 -- project/trading_project/ → empty
PASS
field_runner/** other files changed none git diff HEAD~1 -- field_runner/ → only test_runner_guards.py
PASS
Any mismatch would be PATCH REQUIRED / REJECT ARTIFACT – none.
```

Before: 7bb599f0276a31f919c788a50f7bb51337664aa122d8d5f42e22ef4baf5801c2 (185-line baseline 12c66abe...).

After: c7e6cf8c07b6b2ee54f422aa3b1892ddcb171282493907ad123f42dfe61238e7 — change is docstring (185 → 196 history), closed_manifest_sha256 pin (12c66a... → 899feb...), and assert manifest_lines==196 / ok==196.

B — Live Baseline Authority

Actual guard test_closed_project_manifest_untouched (lines 60-145) now requires:

[Python]

```
closed_manifest_sha256 =
"899febb4a33c984b48176be6a972405fe6be3301faf660d54d53718216090e84"
```

```
...
```

```
assert _sha256_file(manifest_path) == closed_manifest_sha256 # A) exact digest
result = verify_manifest(project_root)
assert result["manifest_lines"] == 196 # B) 196
assert result["bad_paths"] == [] # B) no stale
assert result["bad"] == 0
assert result["ok"] == 196 # B) 196 OK
```

C) 10 artifact seal independent pins (see §E)

Verification on the live tree:

```
closed_manifest_sha256 == 899febb4... PASS
manifest_lines == 196 PASS (196)
ok == 196 PASS
```

```
bad == 0 and bad_paths == [] PASS (no stale allowance exists)
```

Previous 185 baseline not accepted anywhere (see §D) — probe #2 below proves 12c66abe... / 185 now FAIL

Previous 10-performance-artifact seal, if retained, is additional (accepted_artifact_seal dict) — never a patched_seal stale permission. The former patched_seal is retired.

No redundant S0 per-file pins — the MANIFEST digest + full validation is the authoritative complete identity; no alternate acceptance path.

C — Independent Mutation Attacks (10, temporary copies only — /tmp, no repo residue)

Using pristine temp copies (shutil.copytree project/trading_project →

/tmp/mut_*/project/trading_project, field_runner not copied), each attack mutates only the copy, then runs the guard's exact checks:

A) _sha256_file(manifest_path) == "899febb4..."

B) verify_manifest 196/196 OK, 0 bad

C) 10 seal hashes

guard_pass = (sha==closed_digest and lines==196 and bad==0 and ok==196) — for 10-artifact seal, the guard also checks each of the 10 files, but all our mutations that affect seal files also make bad==1, so B already fails; attack 10 is the critical “exact-digest” case where B stays 196/196 but A fails.

Probe Mutation (copy only) Expected guard Observed Result
 1 exact real 196 baseline no mutation PASS sha 899febb4 lines 196 ok 196 bad 0 → guard PASS PASS
 2 real retired 185 baseline replace
 MANIFEST.sha256
 with c82d13a:project/trading_project/MANIFEST.sha256 (12c66abe..., 185 lines) FAIL sha 12c66abe lines 185
 ok 183 bad 2 (old manifest missing 11 new files) → guard FAIL (digest + lines + ok) PASS
 3 mutate one MANIFEST hash for S0 artifact (records.py 1 char) flip hash char for
 market_understanding/records.py line FAIL sha 3bb088b3 lines 196 ok 195 bad 1 PASS
 4 mutate actual S0 records.py without updating MANIFEST echo "# injected" >> src/.../records.py FAIL sha
 899febb4 lines 196 ok 195 bad 1 PASS
 5 mutate accepted performance artifact (core/causal_percentile.py) echo "# injected" >>
 .../causal_percentile.py FAIL sha 899febb4 lines 196 ok 195 bad 1 PASS
 6 remove one MANIFEST line delete last line → 195 lines FAIL sha 513b7f27 lines 195 ok 195 bad 0 (lines
 ≠196) PASS
 7 add unauthorized MANIFEST line append 000... src/trading_system/fake.py → 197 lines FAIL sha 7442f8ff
 lines 197 ok 196 bad 1 PASS
 8 keep 196 lines but replace legitimate path/hash pair swap hashes of first two entries FAIL sha
 60104603 lines 196 ok 194 bad 2 PASS
 9 cause one manifested file to become missing/stale rm src/.../identity.py FAIL sha 899febb4 lines 196
 ok 195 bad 1 PASS
 10 regenerate different MANIFEST to preserve 196/196 on modified project mutate records.py then
 hashlib.sha256(new file) → update that line in
 MANIFEST.sha256
 (so verify_manifest is 196/196 OK but manifest file digest is now 93650221... / 1d6e01de... etc.) FAIL
 (exact digest pin must reject) sha 93650221... (or 1d6e01de... depending on mutation) lines 196 ok 196 bad 0
 → guard FAIL because sha != 899febb4... PASS
 This tenth attack is mandatory: the guard protects this exact accepted baseline, not merely “a clean
 196-line manifest”. All 10 behave exactly as required – no probe remains in repository (/tmp/mut_* only,
 git status clean, git diff shows only guard+report).

Builder probe scripts (if any) were in /tmp per report — none in field_runner/ or project/.

D — Old Authority Search

Search field_runner/ for live references:

Searched Hits in field_runner Classification Live?

12c66abe6f18300f2e00cb5c4befeafe1e3dc67c2ca50bc71db74db9ba8b367d

field_runner/runner_tests/test_runner_guards.py:65 docstring superseded the previous clean baseline
 (185 lines / 185 OK / 12c66abe...) HISTORICAL docstring — explains retired baseline, not an assert NO
 185 as live expected manifest count field_runner/runner_tests/test_runner_guards.py:65 docstring 185
 lines, :77 185-line baseline has NO live... HISTORICAL NO — live assertions are ==196, ==
 "899febb4..." (lines 91,123)

7796a73fc30fc311902d7ba8e033eb1699a73c5db10e824d02653fbdd1681587 0 hits in field_runner —
 NO

183 / 173 / 10 stale field_runner/runner_tests/test_windows_reporting.py:64
 manifest_result={"ok":183,"bad":10} and :95 assert result["ok"]==183 SYNTHETIC FIXTURE —
 formatter test with synthetic manifest_result, not a live baseline authority (test_windows_reporting.py
 is about windows reporting formatting, uses ReferenceReporting synthetic data) NO
 Do-not-cleanup: synthetic formatter fixtures (test_windows_reporting.py:64,95) are deliberately left —
 they test synthetic reporting formatting, not the live guard. BLOCK only if a second live authority
 conflicts with the new 196 baseline — none found.

grep -rn "12c66|7796a|183 lines|185 lines" field_runner hits only the docstring above and the synthetic
 fixture — no live second authority.

deliverables/ retains historical pins (12c66a..., 7796a73...) as documentation of superseded baselines
 — not live.

E — Performance Artifact Seal (10)

Guard retains accepted_artifact_seal (independent pins, extra protection, never as stale allowance).
 MANIFEST already requires 0 stale; the seal adds stricter per-file checks.

Path Pinned (guard) sha256sum on CLOSED project Result

causal_percentile.py

1543794f173e1552435d2d37fd53fa9f7b67ba3c2dfb7c6ddc001f971f0f8654 1543794f... PASS

fvg.py

1df18e553557132788024168ed8b384c338476271cf08c436c23bc01d31dcd4b 1df18e55... PASS

narrative.py

407c4d2f8be00cf66e22232b0da3d88de80a8068dad066aa7ed6a902b0dd24f9 407c4d2f... PASS

hashing.py

f2a64c8e3e098f6149c5d01039122b10a8929e44b430cbd7edf40a10317d5c7d f2a64c8e... PASS

manifest_identity.py

4c88fc44638af9ddd64a6259c86dc3b80fea927097363fe3e87141d35d92414b 4c88fc44... PASS

test_causal_percentile.py

55f2be1bee0b13d60a5e793594e40ab934fa3688764ba1c1f95d237b3e11f228 55f2be1b... PASS

test_fvg.py

471702f7293923748008ce1948b3ee7a1eced7595b16be408f2f6fe22672b05a 471702f7... PASS

test_narrative.py

c9c3000031d046c508d0c82371f6e64839ff8274a6acb139d1e42350afbb0740 c9c30000... PASS

test_research_hashing.py

d9eaf0513c0f58f40cb190a352962ba30058c98818b1a0d88c3df7d53a761e67 d9eaf051... PASS

test_research_manifest_identity.py

15d1a7271c325f38e230bc67de40b0edf3dc3df5bb8e0edf039c97a455485d49 15d1a727... PASS

Seal does not mean “these paths may be stale” — verify_manifest already requires 0 bad; the seal would still fail if any of those 10 became stale (they are also in MANIFEST, and bad==1 would already fail the gate).

F – Regression Tests (same real physical package)

Gate Expected Observed Result

field_runner 36/36 pytest -q field_runner/runner_tests -v → 36 passed in 12.27s (artifact_roles 13, failclosed 6, guards 4, integration 8, windows 5) PASS

full trading_project 1139/1139 pytest -q (82.9s) → 1139 passed – pytest --collect-only -q | awk -F: '{sum+= \$2} END{print sum}' → 1139 PASS

S0 dedicated 67/67 pytest -q tests/test_muf_s0_* -v → 67 passed (records 34, availability 14, identity 13, contracts 6) PASS

MANIFEST 196/196 OK verify_manifest("project/trading_project") →

{'manifest_lines':196,'ok':196,'bad':0,'bad_paths':[]}

test_closed_project_manifest_untouched points to real project/trading_project (via PACKAGE_ROOT + "/project/trading_project" or "/trading_project" fallback) – not a synthetic root – PASS (ran on the same physical package; the 10 attacks used synthetic /tmp copies only for negative cases).

G – Non-Touch (before/after audit)

Guard file SHA before audit c7e6cf8c07b6b2ee54f422aa3b1892ddcb171282493907ad123f42dfe61238e7 → after audit same → unchanged

MANIFEST SHA before 899febb4... → after same, 196 lines → unchanged

trading_project/** git diff HEAD1 -- project/trading_project/ → empty (261/261 files byte-identical per report)

field_runner/** other than guard git diff HEAD1 -- field_runner/ → only guard

Audit created no probes inside repository (/tmp/mut_* only, git status --porcelain shows only untracked deliverables/52_... report which is the builder's report, not an audit probe; no probe*.py in repo)

H – Certification Limit

Acceptance proves only:

the external Field Runner guard protects the exact official MUF S0 CLOSED baseline (899febb4... / 196/196 zero stale, plus 10-artifact seal).

It does not re-certify: S0 semantics, wave logic, predictive support, edge, profitability, Model, Strategy, PnL. Those remain governed by the S0/P1/P2 design audits (already ACCEPTED FOR CLOSURE where applicable) and are out of scope for this guard re-pin.

Proof Summary

Guard SHA: c7e6cf8c07b6b2ee54f422aa3b1892ddcb171282493907ad123f42dfe61238e7 (before